LINQ Fundamentals in C#

Course Overview

Course Overview

Hello. My name is Paul Sheriff, and welcome to my course LINQ Fundamentals in C#. I am a business IT consultant at PDS Consulting with over 35 years of experience creating enterprise applications, stop writing loops to iterate over collections and start using LINQ instead. LINQ can filter and extract data from collections much more efficiently and with less code than you can write. This course covers all the major LINQ operations with over 140 demos using the query and method syntaxes. Some of the major topics you will learn in this course are why use LINQ, using LINQ to select and order collections, filter and find data in collections, extract distinct values, assign values, and partition collections, compare and union to collections, create groups of data, aggregate data using sum, min, max, and average, understand deferred execution and how it maximizes efficiency. By the end of this course, you will have the required skills to use LINQ in your own applications and to go on to learn more advanced topics in LINQ with some other great courses in the Pluralsight library. Before beginning this course, you should be familiar with C#, .NET Core, and Visual Studio 2022 or VS Code. I hope you'll join me on your mission to becoming a great LINQ programmer by watching my course, LINQ fundamentals in C# at pluralsight.com.

Where LINQ Fits into Your Toolbelt

Introduction and Version Information

Hello. My name is Paul Sheriff with Pluralsight. This course is LINQ Fundamentals in C#. This module, Where LINQ Fits into Your Toolbelt. This course is 100% applicable to .NET 6, .NET 7, and .NET 8, C# 10, 11, and 12, Visual Studio Code versions 1.8 and above, and Visual Studio 2022 and above.

Course Goals and Prerequisites

The goals for this course are to understand the advantages of using LINQ. We're going to go through a whole ton of demos, including how to select and order data, search for data, extract subsets of data, find out what is in common within items within collections, what is in common between collections. We're going to learn to join and to do grouping on data. We're going to aggregate data using min

and max and sum and many others and understand how deferred execution works. For this course, I am going to assume you are a C# developer, you are familiar with Visual Studio Code or Visual Studio, you're somewhat familiar with the SQL, Structured Query Language, and that you're new to using LINQ. Now the absolute prerequisites is that you understand C# generics, C# delegates and lambda expressions, and C# extension methods.

What's in This Course and Community Resources

So what exactly is in this course? Well, we're going to learn the LINQ query and the LINQ method syntax side by side. You're going to see examples of both, so you'll understand how both of these work, and I have over 140 demos that we're going to be going through throughout this course, so this is going to be a lot of information, but we're going to do it in such a way that you're going to learn each step along the way. So the way to get the most out of this course is to watch this module because I have some really important LINQ basics in here. Download the starting exercises for each module after this one. And then, you'll be able to just follow along with the demos. As I'm doing them, you can do them. I have some resources for you to check out. You can download the starting samples at my GitHub at this location. You can also check out the Microsoft official documentation on LINQ. They've got some really great information in there. There's also 101 LINQ samples that Microsoft has put together across their whole documentation set. And check out my blog. Just search for LINQ. I've got quite a few articles up there as well.

What Is LINQ and LINQ Integrations?

What is LINQ? Well, LINQ is a SQLlike syntax that you can use in C# and Visual Basic and allows you to query any type of collections that implement IEnumerable T or IQueryable T. Now some of the most common IEnumerable types include arrays, a string since a string is just an array of characters, any list of T, so List of Product or List of Customer, for example, and then other ones like HashSet, Dictionary, LinkedList. All of these are common IEnumerable types. Now if you want a little more information on those IQueryable and LINQ integrations, there's a LINQ to XML course called Working with XML in C#. And there's one for the Entity Framework called EF Core 8: Fundamentals. Besides the LINQ integrations, there's LINQ to objects, so things like using LINQ with strings that we talked about a little bit earlier. LINQ and reflection, so iterating over the properties and methods of a specific object, LINQ and file directories, LINQ to entities, LINQ to dataset. To use LINQ in your applications, all you need to do is add a using statement using System.Linq. What this does is this adds extension methods that are part of the Enumerable and Queryable base classes so that you can apply the LINQ queries to whatever you're going to do.

Examples of SQL, C# Loops, and LINQ

Let's take a quick comparison of SQL, loops, and LINQ. Now, SQL is very similar to the LINQ query language. So we're going to take a look at SQL, looping, and LINQ all side by side. Let's start out with a simple SQL WHERE clause where SELECT * FROM Products where the ListPrice field is greater than 1000, for instance. If we were to simulate that using a C# loop, what we would see here is I would get a list of products, put it into a products variable. I would then create a new list. I would then foreach over that products list. Let's say it's got 50 products in it. And as we loop through each one, we're going to check each one to see if their list price is greater than 1000. And if it is, then we will add that item to our new list. So we end up with now a simpler list than all 50 products. Maybe there's only 13, for example. So that's how we do it using a loop. However, using LINQ, we would say same thing. Let's get our products, so that list of 50 products, for example. And now, we can say var list = from prod in products where prod.ListPrice is greater than 1000 select prod, and then we give it a Tolist to convert it into a list of product objects. You can see it's very similar to the SQL that we wrote earlier. Another example is to select a set of distinct colors from our Products table, which you do using this SQL statement. If I were to do this using a C# loop, I would again get the products, create a new list of strings, loop through each product. And each time through the loop, I would see if that new list contains a product.Color. And if it does not, we will add it to that list so that we end up now with a list of unique colors. We can also do the same thing with LINQ. As you can see here, I would get the list of products. And then from prod in products, select prod.Color, apply the Distinct method, and then do a Tolist to make sure it's a real list of strings. Now we're going to learn how to do these and many others as we move throughout this course. I'm just trying to give you a little bit of an overview right now. And let's take a look at one more example. Maybe you want to get the minimum ListPrice from your products in your table. SELECT using the MIN function in SQL. Give it the field name from your Products table. You can simulate that using a loop. Here we get our products again and put it into our products collection. I'm going to create a decimal variable called ret, and I'm going to set it to decimal.MaxValue. I'm going to loop through each of the products and each time through the loop, I'm going to compare the ListPrice to see if it's less than that ret value. If it is, I'll assign that ListPrice now to ret, loop back up, and continue until I'm out of products. And I end up then with the minimum ListPrice. Same thing expressed in LINQ. I get the products. And now I do decimal value = from prod in products, select prod.ListPrice so it selects that one property, and then apply the Min function. You can see how much more compact LINQ is compared to writing a loop in C#. Now there's a lot of these that you can do. We can select MAX in SQL, and then you can see the corresponding one here using LINQ. We can select AVG and then the same thing applied here in LINQ. We can also do an ORDER BY. So in SQL, we can order. We can sort the data, and we can do it by ascending or descending. I can do the same thing using

LINQ. There's an ORDER BY statement. You give it the property name, and then you can either apply ascending or descending to that. If you want to select just a single column from a table, you can use that column name. Same thing using LINQ syntax as well.

Why Use LINQ and LINQ Operations?

So, why use LINQ? Because it gives us a unified approach for querying any type of object. It could be a collection. It could be XML. It could be a SQL table. We can eliminate looping code, which really cuts down the amount of code you write. We have full IntelliSense support, which is really a great bonus. And we get type checking of objects at compile time. As I mentioned, I have about 140 demos that we're going to be going through. So what are the types of operations that we can perform with LINQ? Well, we can select data. We can project or change the shape of objects into another object. We can do ordering. We can get an element. We can find it. We can do a first, we can get the last, we can get a single one. We can filter using a where clause. We can do iteration and partitioning using foreach, skip, and take. We can quantify. Find out what's within collections. Any of them match this condition? Do all of them match this condition? Does this collection contain a certain item? We can do set comparison. So we can check what's between two different collections to find out if they're equal or if there are exceptions within them. We can do set operations. We can union two collections together or concatenate collections together. You're also going to see how to join two collections together, doing like inner joins and outer joins. We can do grouping where we do group bys and we can even do subqueries. We can extract distinct sets of data, and we can aggregate data as well. So tons of things we can do with LINQ. These are all the things that we're going to be doing throughout this course. In this module, we learned that LINQ is a SQL-like syntax for C# or Visual Basic. It can be used with many different types of collections. We can search, we can order, we can group, we can join, we can aggregate. Many different operations can be performed with LINQ. And we can integrate with XML, databases, and many other types of objects. Coming up next, we're going to start out using LINQ to select data within collections. I hope you'll join me for the next module.

Use LINQ to Select Data within Collections

The Console Application Used for LINQ Samples

Hello everyone. Paul Sheriff here with Pluralsight. This module is Use LINQ to Select Data within Collections. The goals for this module are to provide an overview of the console application that I'm using and that you should use

to follow along as I move through the demos. We're going to talk about selecting all data, selecting specific columns from data, and building an anonymous class. Let's get started. Let's begin by taking a look at our overall application. It is a console application, so it's very simple. It's using .NET 6. It's a console app that runs each sample as we build them. We have some additional class libraries attached to our console application. The first one has some classes that we're going to visit as we move throughout the course, and they're just classes that help us query in various ways. And then we have a data layer class library. Now this class library is where I have some product and sales type classes that hold product and sale data. And then I have some repository classes that help us build collections. These are hardcoded collections. I didn't want to have to have you worry about connecting to a database or anything, so I've created some just hardcoded classes with data that will be fed to us that we can then query.

The Sample Entity, Repository, and View Model Classes

Let's take a brief look at some of the classes we're going to be using. The first one is a Product class. This represents a single product. And each property listed here would actually have a get in a set, but I have eliminated those on the slide here just to keep things simple. But you can see we have a product ID, we have a name, we have a color, we have a standard cost, a list price, and a size for each product. We're then going to have a ProductRepository class. Now this is a class that we're going to use to retrieve a collection of product data. So this method called GetAll would normally be retrieving all the products from a data source. I'm going to be using hardcoded values for this course. So as you can see, I'm just creating a new product, and then I fill in all of the properties with some data. There's only one listed here on the slide, but obviously, there would be a lot more in the actual class itself. Another class that we have in our class libraries is ViewModelBase. Now this is a base class that's going to be used by the SamplesViewModel class, which you're going to use to write all of your queries. Now it has a method to retrieve the set of product objects. It simply calls that ProductRepository.GetAll. It has a method to retrieve a set of SalesOrder objects as well. So we have a SalesOrder, just like a Product class. We have a SalesOrder repository to get a list of SalesOrder objects. Finally, we have a whole set of overloaded methods called Display. And the display is used to display a product, a list of products, a sales order, a sales order list, as well as many other different kinds of lists, string, decimal, and things like that. Now the class you're going to be using and what I'm going to use to illustrate all of the LINQ functionality is called the SamplesViewModel. It inherits from the ViewModelBase. And we're always going to have two methods, one method to illustrate the LINQ query syntax and one method to illustrate the LINQ method syntax. So for each of these methods, what we're going to have is a method that returns a set of data like a list of products or list of sales order, maybe even just a

string or a decimal value. We're then going to build a collection to query. So we're going to use that GetProducts method to fill up our products collection here that's identified List of Product. I then have a variable that's going to hold the results. I'm then going to write the query using whatever LINQ methods I'm going to illustrate, and then hopefully you're going to be following along with that as well. And finally, we return the results out of this method. One last file you're going to be interacting with is Program.cs. This is the starting point for a console application. What we're going to do in here is we're going to bring in the LINQ samples namespace, which is what I have named my project. We're going to create an instance of our SampleViewModel class. We're going to call the method that we just have entered our LINQ query into, and then we will display the results in the console window.

Select All Items Using LINQ

Now that you have a feel for all the classes in our sample project that you can download and use to follow along with me in this course, let's start out and show you how to select all items using LINQ. Once you download the samples, locate the particular module on, for example 03 and that's the Select one. There'll be two folders underneath here, Finish and Start. Finish has all of the finished samples after I've done them in the video. Start is where you can follow along with me. So what you would do is go to Start and you would choose either the LINQSamples.code-workspace if you're using Visual Studio Code, or you'll choose LINQSamples.sln if you wish to use Visual Studio 2022. I'm going to double-click on the code-workspace file and bring it up in VS Code. Here I am in Visual Studio Code. At the top, I have my LINQ samples workspace. Underneath that is my console application. I then have LINQSamples.Common, and here you can see the ViewModelBase class that I talked about. Under LINQSamples.DataLayer, under that project, there's a CompositeClasses folder, which we'll take a look at those a little bit later. We have the entity classes, that Product class and a SalesOrder class as I mentioned. We also have the RepositoryClasses folder. And underneath that, we have our ProductRepository and our SalesOrderRepository that we talked about. And then there's a couple of additional ones, and we'll talk about those as we go into this course. Bring up the SamplesViewModel class. And inside of here, you'll find, GetAllQuery. What we're going to do is we want to populate the list on line 14 with the results from the list of products that we get from the GetProducts method call. So we would say list equals. Now I like wrapping mine within parentheses. Be careful with Visual Studio Code. It doesn't always give you the correct syntax. It'll try to fill it in. So just hit Escape there, and you can then say from prod. So make up a variable in and then our collection, products. So we're saying from this variable in this products. And what it's going to do is take each item. Think of this as a foreach loop. It's going to take each item in the products collection and give it to prod. Then you select prod, and that selects the

complete object. And then what I'm going to do is apply a ToList here. By applying the ToList method, it takes it from what's called an IEnumerable and casts it into a list of product objects because that's what I've defined the variable list to be. Once you've written the syntax, if you go over to Program.cs, make sure it's calling the method that you just wrote. So we just wrote the one GetAll query. So make sure this says GetAll query, and then you're allowed to go ahead and run. And I've set mine up so it displays in an external console window. You'll see the results of each of our products. You'll see the name. You'll see the ID, color, the size, the cost, and the price. Now the way that I set this up to be an external editor is under the VS Code in the launch. I set here on line 17 console externalTerminal. That is not the default, so you will have to change this if you build your own console application. Let's do one more sample here. We'll come down here to the GetAll method. We're going to do the same thing, same result. But what we're going to do is use the method syntax. So now we're not doing the from and the in and all of that. What we're going to just simply use is the select method. You have a variable for this lambda expression that passes it each time through the loop, the select. Think of the select is looping through, passing in the product object each time to the first parameter there, prod. Passes it to the other side, which simply says okay, return prod. Go ahead and do your ToList. Take your method name here. Go over to Program.cs. Copy it there. Go ahead and run, start debugging, and you should get the exact same results in your console window. Now let me point out a couple of things about the methods that I'm writing here for the course. There are many methods within the SamplesViewModel class, and that's why I had to set the list variable to new so that all methods would compile. Once you set the list of the results of the query, you can eliminate the equals new portion. I just did this just so that we can get things to compile. Also, in a real-world scenario, you can actually eliminate the list variable. You can simply return the results of the query. The only reason I'm doing this is in the course I will be showing you the results of that list variable in the debugger, so it just makes it easier to add the variable while we're doing this course. One more thing I wanted to point out is why am I showing both the query and the method syntax? Well, there's a couple of reasons. Sometimes the queries must be done at the method syntax because the query syntax can't express everything that's available in LINQ. Also, you may be reading an article or a blog or some samples out on the internet somewhere and the author may be used one or the other. So, again, very good that you should know both methods so that you're able to read both ways of doing this.

Select a Single Column

So this is what we're going to be doing throughout the whole class. We're going to be coming into a prewritten starting point that I've already written for you, and you're going to simply enter the LINQ query. So let's take a look at maybe we just want to get a single column or a single property from that list and not a product

object. Let's take a look at how we do that. Move down to the GetSingleColumnQuery method. In this one, we're still getting our products from the GetProducts method. But on line 46, we're creating a list of string. The reason why we're doing that is because we just want to select a single column. So I'm going to add the range of data that I get from prod in products select prod.Name. Now again, we're pointing out here that LINQ gives us IntelliSense because this is compiled code, which is wonderful. So if I say I just want to select prod.Name, Name is a string variable, it's going to create a collection of string objects, and I'm going to add that range of all those that come back to my list of string. Again, we go ahead and we copy this into our program. Make sure we're calling the right method. And then let's go ahead and let's set a breakpoint right here this time. And we'll go ahead and run this with debugging. So when we come in here, we can see under products, we can see all of the products that are coming back. There are 40 of them that I hardcoded in that product repository. We're then going to create a list. Now this has no items in it right now. So what we're going to do is we're going to say, okay, let's loop over this product's collection, put it into the prod variable each time through, and then select from that prod variable just the name property. And it creates a list of strings, which I'm then adding that range of strings to the list. So once I continue on, so let's see, it's just iterating over each one. So as I click on this and say continue, it's just going over each one. Let's go ahead and move the breakpoint down here. I'll press F5, and now we can see our list is now simply a list of strings that I can then put out into the console window. Let's go back over here one more time. Go down to the single column method approach, list.AddRange just like we did before, products.Select, prod prod.Name. Now, by the way, you don't have to— This can be any name you want. So I could simply keep it a little bit more succinct and just do p for product or c for customer or s for sales. Whatever you want to put there is fine. This is simply a variable name. But just to prove it, let's go ahead and copy and paste this now. So we use this method. Let's restart. It should save everything. We now run the same thing using the method approach. We get the exact same results.

Get Specific Columns to Load into a Product Object

When moving objects from one list to another, you may only need to get a few properties. Let's take a look at how to accomplish this. Scroll down in the SamplesViewModel class and find GetSpecificColumnsQuery. On line 78, we're going now create another list of product objects. So the list is going to be equal to from, make sure you hit Escape there, prod in products. I'm going to go ahead and hit Enter here so I can put things on to the next line down. I'm going to select. But instead of just doing a select prod, I'm going to actually select new product. So I'm going to create a new product object here on the fly. And then it's going to give me the IntelliSense because that's what Visual Studio Code does. And I'm going to say, let's grab the product ID and put it into the product

ID. Let's put into the name prod.name, and let's put into the size prod.Size. When I'm done with this, I'm going to do a ToList because I need to convert all of this to a new list. So what I've done is created a new product object, but I've only filled in three of the properties. Go ahead and copy this method over into Program. Go ahead and run this. And when we see the console window appear, what we're going to find now is that we can see the name and the ID, and we see the size. But the color, the cost, and the price are all not filled in. So this is an example of just getting specific columns. So if we go ahead and come back over here to the specific columns method approach, let's go ahead and write this one. It's very similar. So what I'm going to do here is I'm going to go ahead and just kind of copy some of this, and then we'll just change it. So if you copy and paste this down, what we can do now is we can simply say products.Select prod. And now, we do our arrow syntax here, but we're going to create the new product there. Very simple in a lot of cases to move something from the query syntax to the method syntax. Now, Microsoft kind of always recommends you use the query approach. So it's up to you which way you want to go. They say use the query approach because that way they have the basically the ability to change what goes on underneath the hood. But recognize that the query approach actually gets compiled into the method syntax when it's finally done. Either way you want to write it I think is fine. I have not hit too many instances where from version to version they've changed a lot of this. Usually, what they do is they add on additional methods. All right, make sure we're calling the GetSpecificColumnsMethod here. Go ahead and start the debugging, and we should again end up with the exact same result as we did with the query approach.

Build an Anonymous Class

Now the next thing we're going to do is instead of using a product class, we're going to create an anonymous class. And that way we can give the new properties whatever name we want. Now this is a little bit of an edge case. I'm not a big fan of anonymous classes because they're temporary. They're locally scoped. You really can't pass them around unless you want to use the dynamic keyword, but then you have all these other issues. So, this is something I'm going to show you how to do. Know that I really haven't used it that much at all. I can't even think of a time when I've used it. But maybe it might come in handy, so let's take a look at building an anonymous class using LINQ. Now, for this one, I'm still getting my products by calling GetProducts. But now I'm going to connect up a string builder because, remember, I can't really return this anonymous class from this method because it's locally scoped. So I'm going to do var list, and it's going to be from prod in products. I'll then do a select new, but I'm not putting in the name of a class. So this makes it anonymous. So I'm going to just create my own variable name. I'm going to call it Identifier, and I'm going to assign that to the ProductID. I'm going to create ProductName this time instead of just Name, and

I'm going to grab that from my prod.Name. And then I'll create ProductSize prod.Size. So what I've done is I've now created an anonymous class. If we hover over list, you can see it's an IEnumerable of this kind of 'a, which stands for anonymous type. But it actually tells you what the thing looks like. It's an int, it's a string, and it's a string. Now, what I'm going to do in this case is I'm going to go ahead and uncomment this code, this foreach that I already had prewritten. That'll allow me to loop through that list. And then each time through, I'm going to do an sb.Append, my string builder .AppendLine. And I'm going to put in that product identifier, that product name, and that product size into this string builder. And I'll return this from this method. Go ahead and copy and paste this now into your program. When we call the display, it's going to be a string. So luckily, I've got one of the display methods. Down in the ViewModelBase class here, we'll display a string. So let's go ahead and run this, and there we go. So there is our anonymous class with the new properties, identifier, product name, and product size. Go back over to the AnonymousClassQuery, copy this query syntax, and paste it down into the next method, which is where we're going to write the method syntax. Uncomment out the foreach. And then this time, all we're going to do is we're going to change this to be products.Select, add in our prod with the arrow syntax, and it's going to be equal to the new anonymous class that we're creating here. Go ahead and grab this method name. Paste it into here. And if we've done everything correctly, once again, we should see the exact same results as the query syntax. In this module, we saw that the query syntax is very readable, if not a little verbose; whereas the method syntax is very compact. With LINQ, we can select single properties, we can select multiple properties, and we can learn to project new columns using anonymous classes. Coming up next, use LINQ to order data.

Use LINQ to Order Data

Sorting Data on a Single Field

Hello. Paul Sheriff with Pluralsight. This module is Use LINQ to Order Data. The goals for this module are to learn how to perform the ordering of data or the sorting of data in collections, to show you how to do it in ascending and descending order, and how to use two different fields for ordering. Let's get started. When sorting data with LINQ, the query syntax uses an orderby clause, all one word. The method syntax uses the OrderBy method. Let's take a look at our first demo here of ordering data. Load up the start folder in Visual Studio Code for module 4, and open up SamplesViewModel and locate the OrderByQuery method. So we're doing the same thing. We're going to get our list of products by calling GetProducts, and then we're going to create a new list with the data sorted. So we're going to write our code here to do from prod in products. And then we will select prod. Now that's what we normally do. However, what we're

going to do now is we're going to add something in between the from and the select, and that's going to be the orderby. And then you just choose which property in our product object you want to sort by. Let's go ahead and sort by the name. After we've done that, you add the ToList, and we now have the data sorted by the product name. Go over to Program.cs. Make sure OrderByQuery is the method that's going to be called. Click Run, Start Debugging, and we should see the console window appear. And now if you look up, you scroll up to the console window, you'll see that now it is sorted in name order. You see the As, then the Hs, then the Ls, and then Lo, Ms, and then finally the Ss. Go back to the SamplesViewModel class, find the OrderByMethod method. And now, let's write the equivalent method syntax where we do products.OrderBy prod.Name .ToList. Now we don't need the select necessarily. The select was kind of a little bit superfluous, but I wanted to show you the select because we're going to do some other things with that later. But this is the simple method syntax. Once again, copy this over to the Program, go ahead and run, and you should see the exact same data coming back in the exact same sorted order.

Sorting Data in Descending Order

To sort by descending order using the query syntax, you add descending after the field name that we are descending on. For the method syntax, we use the OrderByDescending method. So let's take a look at a demo of ordering data in descending order. Let's take the query that we just wrote, copy that to the clipboard, move down to the OrderByDescending query, and paste that in. Now right after prod.Name, go ahead and add in the keyword descending. So this will make things go in reverse order. Copy the method over to Program and go ahead and run. And now, the last one we see on the screen should be the A. And as we scroll up, you see it's going up the alphabet. Let's go ahead and do the same thing here. Let's grab in the OrderByMethod that we wrote, copy that, paste this down. But now change OrderBy to OrderByDescending. So the method syntax and the query syntax are just a little bit different here. But again, as usual, the end result is exactly the same.

Sort the Data Using Two Fields

Let's say you want to order by two separate fields. In the query syntax, we separate those fields using a comma, for example orderby prod.Color, and then you could add descending, comma prod.Name. Now the method syntax is a little different. We're going to add the second column by chaining together a ThenBy method or a ThenByDescending method, whichever one is appropriate. So let's take a look at a demo of ordering data by two fields. Copy the code that you just wrote in the OrderByDescending query and locate the OrderByTwoFieldsQuery and paste that there. What we're going to do now is we're going to do an orderby

prod.Color, and we're going to do color in a descending order. And then we'll add on the prod.Name. Now just so you know, you can add the ascending keyword. That's fine. It is unnecessary because that is the default. But if you wish to be very explicit, go ahead and add it. Now that we've added that, let's go ahead and OrderByTwoFieldsQuery. Copy that to the clipboard. Paste that here. So that's the sample you're going to run. When the console window comes up, if we go all the way to the top here, you'll see that we have the first two. The color is white, w being towards the end of the alphabet, it comes first. Then you can see we have some silvers, and let me see if I can scroll this a little bit easier. So we see the silver. Now all of those are still Hs. We have red, so HL. But then we have L. So it is going the name of the product is actually going in ascending, even though the colors were going in descending. Go back to the SamplesViewModel and locate the OrderByTwoFields method, and let's go ahead and write the method syntax. Now what we're going to do here is we're going to do an OrderByDescending, and we'll do prod.Color. And then we add the ThenBy. Let's go ahead and put this on the line here. So we'll do a .ThenBy. So this is how we actually add on a second or a third or a fourth column. You just keep chaining together the ThenBy. Or there is a ThenByDescending as well. So there's our method syntax to accomplish the same thing we saw for the query syntax. Once again, copy this and run, and we see the exact same results we saw with the query syntax.

Sort Two Fields Descending Using the Method Syntax

Now just to make sure you've seen all the combinations of things. Let's go ahead and do the method syntax of ordering data by two fields in descending order. Take the last method syntax that you just wrote. Copy that down into the last method here, OrderByTwoFieldsDescending method. And this time, we're going to do OrderByDescending prod.Color, ThenByDescending prod.Name. Go ahead and copy this over and run this. Now what you'll see here on the red, color red, you'll see the sport is first and then Ms come after that. So the S being later in the alphabet comes before M. In this module, we saw that we can order data either ascending or descending. Now we also notice that the query syntax does get a little verbose in here; whereas the method syntax, we're simply chaining methods together, things like OrderBy, ThenBy, OrderByDescending, and ThenByDescending. Again, up to you, which method you want to use either query or the method syntax. But just know that they both basically get translated down to the method syntax in the end. Coming up next, use the LINQ where clause to filter data. I hope you'll join me for the next module.

Use the LINQ Where Clause to Filter Data

Using the Where Clause

Hello. Paul Sheriff here with Pluralsight. This module is Use the LINQ Where Clause. The goals for this module are to use the where clause and understand how it works. We're going to see how to find items or a single item in a list collection. We're going to find out how to use and and or in a where clause and talk about building your own custom extension method. Let's get started. Just like we saw doing ordering, when we're filtering data, the query syntax uses the where clause, and the method syntax uses the where method. Let's take a look at a couple of examples of filtering the data. So in a query syntax, we would do from prod in products where prod.Name.StartsWith S, for example, and then select prod. So you're selecting the whole product, but only those now will be returned where the first letter of the name property starts with S. To express the same thing in the method syntax, it would be products.Where prod arrow syntax prod.Nam.StartsWith S. So let's take a look at a demo of writing your first where expression. Open up the project in the start folder for module 4. Locate the WhereQuery, and then let's write our queries. So from prod in products. And then let's go ahead and add the where prod.Name StartsWith and then use S. We then select prod and, of course, convert it to a list. Copy your method name here to Program.cs. Make sure it's the same one, and let's run and take a look at what happens now. If you remember, there were 40 products total. But now when we run this with a where clause, we only get three because it looks at that first letter of the name property and only matches against those. Let's go ahead and go back and write the same thing now in the where method, list = products.Where. I want to go ahead and shorthand this now. Instead of writing out prod every single time, I'm just simply changing the variable name. Not a big deal. You can choose to do it either way. But here is now the appropriate method syntax. Again, copy and paste that over, and let's try this out. And hopefully we will see the exact same results of three products.

Using the And (&&) Operator

If you wish to filter your data on two or more fields, then you can add the and operator. So here's an example of using the C# and operator within our where query. So from p in products where p.Name starts with an L and p.StandardCost greater than 200, for example. So that's the query syntax. And then, of course, we have the exact same thing expressed here in the method syntax. So let's take a look at a demo of the wehre expression with two fields. Copy the query that you wrote in the WhereQuery method and paste that down into the method called WhereTwoFieldsQuery. Now what we're going to do here is we're going to change this to be a StartsWith L and prod.StandardCost greater than 200. So now we're using the C# and operator to filter based on two fields. Let's go ahead and copy this method name over to our Program and let's run this. Now what we get back are six products. So if we take a look here, we can see that all six of these match where the first letter of the name property starts with L, and the cost is all greater than 200. Back in SamplesViewModel, locate the Where method and copy that

query that you wrote. Locate the WhereTwoFieldsMethod and paste it in. Change this to be an L and then do the StandardCost greater than 200. So let's go ahead and take this method name and paste it in here. Run this. And we should see that we get the exact same data that we did when we were doing the query method.

Custom Extension Methods

Let's take a look at creating our own custom extension method where we create a method that returns an IEnumerable T. That way we can use this method on the query instead of like a where, for example. So we could do from prod in products select prod .ByColor and then we pass in the parameter like red. Now this is exactly like doing a where clause but it allows us to do things that are could be maybe more complicated. What we do is we create a static class that has a static method in it. And if you'll notice here, we're using the this keyword to key to the type of object, which, in this case, is an IEnumerable of Product, IEnnumerable of T, right? So if you're familiar with extension methods, this is a very common thing we do to extend classes that Microsoft has built or other people have built. We then also pass in any other parameters, in this case a color. And then, we invoke the where method to filter the data. So query is what we passed in as the parameter, and we return query .Where, pass in p, p.Color is equal to the color that was passed in in the parameter.

Demo of a Custom Extension Method

All right, let's take a look at a demo of using a custom extension method for filtering. Open up the LINQSamples.common project. Underneath the HelperClasses folder, there's a ProductHelper class. This is a static class that has this public static IEnumerable product method called ByColor. We know it's an extension method because of the keyword this before the first parameter that says when you're using an IEnumerable of product, that query you can add on this method. We also pass in a second parameter, color, which we then use on line 9 here where I do query.Where arrow syntax p.Color is equal to color. So we're adding this where clause to the query that's passed to this method. If we go back to the SamplesViewModel now and we type in list from prod in products select prod, we can now say .ByColor. And there it shows up in the IntelliSense because it can recognize that it's an extension method for something of IEnumerable product. And then all I have to do is pass in whatever color I'm looking for. Cast that back as a list, which executes that query now and puts the value back into the list variable. Let's take our WhereExtensionQuery, copy that method name over to Program, and let's run this. And we can now see that we're only getting total products of 18, and all of those products in this list have the color red. Go back to the SamplesViewModel, locate WhereExtensionMethod, and we can write the same thing using the extension ByColor Red .ToList. We can

copy this over here and just make sure that we did everything correctly, and we should get the exact same results as we did before where total products 18. In this module, we learned to use the where clause to filter our product collection. We also saw how you can use the C# and operator to use two or more fields, and you could also use the or operator as well depending on your needs. We also saw how you can write your own custom where filter using an extension method. Coming up next, select a single piece of data from a collection. I hope you'll join me for the next module.

Select a Single Piece of Data from a Collection

The Methods for Selecting a Specific Item

Hello. Paul Sheriff with Pluralsight. This module is Selecting a Single Piece of Data from a Collection. The goals for this module are to find a specific item in a collection, and then we're going to learn how to search for that specific item using a few different methods. We're going to use First or FirstOrDefault, Last, LastOrDefault, and the Single or SingleOrDefault methods. Let's take a look at each of these methods and how they work. The first one, FirstOrDefault, you pass in a lambda expression of what you're looking for and optionally a default value. What it does is it searches forward in the collection, and it returns null if no value is found. Or if you supply a default value, it will return that value instead of null. LastOrDefault, same thing. You pass in a lambda expression of what you're looking for, as well as an optional default value. It searches backwards in the collection. So it starts at the end of the collection and searches backwards, returns null if the values are not found based on the expression. Or if you supply the default value, it will return that. Now, some of the methods will actually throw an exception if you don't find anything based on the expression. So for example, First, you pass in your lambda expression of what you're looking for. It searches forward in the collection and finds that first one that matches the expression and returns that. If it doesn't find it, it throws an exception. Same thing with Last. Again, you pass in the expression of what you're looking for. Searches from the end of the collection backwards. It throws an exception if no values have been found. If you're familiar with your collection and you know there's only a single item you're looking for, you can use the Single method, passing in your lambda expression of what you're looking for. It will search through the collection forward. It'll throw an exception if the collection is null or if more than one element is returned. So this one, it truly only wants a single value to match in that collection. The other option is SingleOrDefault. Again, you pass in the expression and optionally a default value. Now, this will throw an exception if the collection is null or more than one element is returned. Otherwise, if it doesn't find anything, it won't return a null or whatever default value you supplied. Because we're going to be working with LINQ methods that can now throw an

exception, we're going to need to wrap things into a try catch. We're going to change our Program.cs. We're still going to create an instance of our view model in Program.cs. We're now going to wrap the method to try out within the try block. Then, we're going to have a catch ArgumentNullException. So this would happen if the product collection was null. And if we get an InvalidOperationException, if we're calling the First or the Last methods, that means that no item was found that matches the criteria. And if we're calling any of the Single methods, it means that multiple values were found. So what we're doing is we're simulating you're making a call to some sort of LINQ operation that you know could throw an exception. You need to make sure you have a try catch in your application.

Search Forward for an Element Using First()

Let's take a look at our first demo, searching forward for an element using the First method. Let's start out with our FirstQuery method. And what we're going to do is we're going to write our typical from prod in products, and we're going to select the prod. And then I'm going to say, let's find the first. The first is going to be based on the color equal to red. So that's my criteria that I'm trying to find. We're trying to find the first product that matches this criteria. It's going to search forward through the collection of products, and it's going to give me back that value, which is a single product. So let's go ahead and we'll make sure the FirstQuery is the one that we're going to run right here. And we can see we get the first one in the list where the color is red. But what if we want to now try out something where it is going to fail? So let's put in purple. I know I do not have any items in my collection that are purple. So let's go ahead and try that. We can comment this one out and let's run this one more time. And now, when we get the display screen, it's going to show us that InvalidOperationException.
The sequence contains no matching element. So this is your clue that you tried to search for something and there's none that match that criteria. So just to be very succinct here, let's go ahead and do the same thing. We'll write our method syntax here. So our method syntax looks very similar. We just don't have to do the query portion.

Search Forward for an Element Using FirstOrDefault()

Let's look at a demo of searching forward through a collection, and we're searching now for an element using the FirstOrDefault LINQ method. Go to the first query method that you wrote. Copy the query. Go down and find the FirstOrDefault query and paste it in. Then, instead of First, call FirstOrDefault. Now this one will never raise an exception. What it will do if it does not find anything is it will give you back a value of null. So let's go ahead and try this one out. So the FirstOrDefault for a good condition gives us back that same product we saw

before. Let's now go and change this so that it now searches for something that is not there. And let's go ahead and now run this. And we can see we get a big fat nothing because it returns null by default. One of the options I mentioned is that you can pass a second parameter to the FirstOrDefault. And when you do that, that is what you want to return instead of a null. So I could create a new product, and maybe I set the ProductID equal to -1 and maybe I set the name equal to NOT FOUND. So by adding this second parameter here, this says if you don't find it, return this instead of a null. Let's go ahead and run this. So for the good scenario, we're still getting back our product. Let's now go ahead and add this here on the bad scenario like so. Comment out the first one. And now let's run this again. And there you can see now, the product NOT FOUND, ID -1. Because it could not find anything, that's the default value it returns. We can do the same thing for the method syntax. Change this to FirstOrDefault, and you still have the same option where you can pass in that second parameter.

Search Backward for an Element Using Last()

Let's now use the Last method to search backwards for an element through our product collection. Locate the LastQuery method and write the syntax you see here on lines 134 through 136. Very similar to the same you wrote for First. It's now we're just using the Last method. So Last is now going to search backwards. Take this method and let's run it. Now we're still finding a product object here. It's still red. But if you notice, this one's ML Road Frame 60; whereas the other one that we are finding started out with the HL. So remember, it's searching backwards through the collection now. Now just like the other one, we can also write this one to fail. So let's go ahead and put in a color that does not exist. And just like the First method, this one will throw an exception. And we'll catch that exception, and then we'll display it here. Now, obviously, in a real-world application, you're going to do something else with that exception. Probably go on and perform something else or tell the user that you can't find what they were looking for. We can also go and write the same code in the Last method. And so on line 57, we can do the method syntax here where its value = products.Last and then pass in your lambda expression of what you're looking for. And that is the method syntax now for using Last.

Search Backward for an Element Using LastOrDefault()

If you do not want to deal with exceptions, which I typically don't, I'm going to typically use the FirstOrDefault or the LastOrDefault, and LastOrDefault is the one that searches backwards. Let's take a look. Locate the LastOrDefault query method and write the query you see here on line 173, value = from prod in products select prod.LastOrDefault and then the same lambda expression that we have been using. So if we run this, we should see we get the exact same

results as we did with the LastQuery method. We'll get that last one in the collection, which is the ML Road Frame. Now, it will also give us back a null if we choose an item that does not exist. So if we put that in, comment this one out, and we run, we will now get a null value display because it can't find anything with purple. And just like you can do with the FirstOrDefault, you are also allowed to pass this second parameter. So I can use the same code we did before where I create my own product with its own product ID and its own name that will be displayed if nothing comes back from finding that purple. And just to be complete, let's come down here to the LastOrDefault method and write the method syntax here, value = products.LastOrDefault using the same lambda expression we saw before.

Searching for Only One Element Using Single()

Let's now talk about the Single methods. The Single methods, they are going to return a single element, and there should only be one element that it finds in the collection that matches the criteria. It will throw an exception if that element is not found. It will also throw an exception if multiple elements matching that criteria are found. So, let's take a look at our first demo here of searching for only one element using the Single LINQ method. Locate the SingleQuery method and write what you see here on lines 214 to 216, value = from prod in products select prod .Sngle. Now instead of looking for a color, which we know are multiple values there, let's search on that ProductID. Now, I know my data and I know the ProductID is a unique value. So I know that the single method is going to work when I use this criteria. Let's test that out. And there you can see we've gotten that one single element. Let's now go back here and take a look at what happens though if I were to try to select and get multiple values back using Single. So we comment this one out, and let's go ahead and run this. And what we see is we get the System.InvalidIOperationException because the sequence contains more than one matching element. And that's exactly what we're looking for. That is exactly what it's expected to do. The other thing that we could do is check to see if we made that list null. Now actually, any of them here that I've done will throw an exception, and that's why I've got the nice ArgumentNullException around in the program. But what we're going to do is I'm going to go ahead and set products equal to null right here. So when I do that, now it's going to get an ArgumentNullException that's going to be thrown. So literally, any of these could actually fail even for using the FirstOrDefault if the collection is null. I just thought I would show this here because we're talking about so many different types of exceptions at this point. So, almost any time you're using LINQ, you should probably be checking that collection to make sure it's not null prior to doing any sort of searching. And once again, the method syntax is very similar. So locate the SingleMethod method and write, as you see on line 245, value = products.Single and then use your lambda expression that you know will only return a single element.

Searching for Only One Element Using SingleOrDefault()

Let's now take a look at using the SingleOrDefault LINQ method. Locate the SingleOrDefaultQuery method. And starting on line 262 through 264, write this query. So we do our typical value = from prod in products select prod and then do the SingleOrDefault, again using the good path. We know that there's only one product ID, so this is going to work correctly. Let's go ahead and make this the method to run. We run this and we should get back our single product object. But let's take a look at some of the other things that we could do that could cause issues. So for example, if we want to check for finding multiple values, we would do the SinguleOrDefault prod.Color equals red. Now we know that that's going to find multiple values, so let's start this one. And yes, we get our InvalidOperationException that says the sequence contains more than one matching element. Let's come back down here and check for what happens when we clear all of the items in the products collection. So let's go ahead and run this one now, and let's comment that out first. And let's go ahead and run this. Now, in this case, the product's collection is just empty. So when we try to search, nothing comes back. So, SingleOrDefault says simply return a null. But what about if we were to say we don't want a null? Well, we can add that second parameter, just like we did on the other one. So we do a new product here. We can say ProductID is equal to -1, and the name can be NO PRODUCTS IN THE LIST or something, whatever you want to call that. And there we go. Let's comment this one out, and let's run this. So again, I'm clearing the products, and then I'm running this, and now we get the default value. So let's do one more thing just for completeness, and let's write the same thing for the method syntax. And that's what it would look like right here, value = products.SingleOrDefault and then using the same lambda expression that will just return a single item.

When to Use Which Method

So now I've shown you all these LINQ queries, but which ones do you use and when do you use them? So I'm going to do a comparison between the First and Last methods versus the FirstOrDefault and the LastOrDefault methods. If you expect the element to be present in the collection, you can use First or Last. But if you're not sure if that element is present, then use FirstOrDefault or LastOrDefault. And that has to do with exceptions because the First and Last is going to throw an exception if something is not found. So if you want to handle that or maybe rethrow an exception if something's not found, you could use First or Last. Me, I'd rather not deal with exceptions, so I really prefer FirstOrDefault or the LastOrDefault. Also, this gives us back a null or some other default value, and I find it easier to check for null than actually check for an exception. Well, how about First versus Single? Well, these are pretty similar because if you expect the

element to be present, you can use First or you can use Single. If you want to handle or throw an exception if it's not found, you could use First, you could use Single. Here's where it really matters. First is only going to search until it finds the element, then it stops. Single, however, has to search the entire list every time. And the reason why is because, remember, it has to check to see if there are multiples so it knows whether or not to throw that exception. So what does that mean? It means that First is faster than Single. Single is always going to be slower because it always has to search. If you've got a collection of 10,000 items and first, you're looking for the first thing and it happens to get it at 5, it's done, Single still has to go through all the rest of the collection. So what about SingleOrDefault versus FirstOrDefault? Well, if you expect the element to be present, SingleOrDefault works. If you're not sure if the element is present, use FirstOrDefault. If you want to handle or throw an exception if something's not found, SingleOrDefault is fine. But if you don't want to handle exceptions, you're really more interested in getting back a null or your own default value, take advantage of FirstOrDefault. Now again, SingleOrDefault must search the entire list every time you call it; whereas FirstOrDefault searches only until it finds the element. That means that SingleOrDefault is always going to be slower than FirstOrDefault. So again, I would typically use the FirstOrDefault. In fact, I can't even remember if I've ever used the Single methods. In this module, we talked about locating a single item in a collection using First or FirstOrDefault, Last or LastOrDefault, Single or SingleOrDefault. We saw that there's an option to supply your own default value, that sometimes you need to catch exceptions, or sometimes you need to check for null or that default value depending on the method that you're calling. Now, I always like using all the OrDefault methods so that I can avoid anybody throwing exceptions. To me, throwing exceptions are not something that you want to use as a regular way of programming. Those should be exceptions. Coming up next, retrieve specific items using Take, Skip, Distinct, and Chunk.

Retrieve Specific Items Using Take, Skip, Distinct, and Chunk

Using the Take() Method to Extract Data

Hello. This is Paul Sheriff with Pluralsight. This module is Retrieve Specific Items using Take, Skip, Distinct, and Chunk. The goals for this module are to perform partitioning operations where we might take a certain amount of elements from the front, or we might skip a certain amount of elements from the front of a collection. We're also going to see how to get a distinct value from a collection. And we're also going to learn to chunk a collection into smaller sets. Let's get started. Let's dive right into our first demo with learning the Take

method, which allows us to take a specific amount of elements from the beginning of a collection. Open up the start project for this module. Locate in the SamplesViewModel class the TakeQuery method. What you're going to do is you're going to write the query that you see here, so this is very similar to what we wrote before where we do list = from prod in products orderby prod.Name select prod and then .ToList. Now if we just run this, we see we get 40 products just like normal. If you scroll all the way to the top, you'll see that it's ordered by name. So remember that there's an A and then there's an HL Mountain Frame. Let's now go back and let's add on .Take, and let's just take the first five. So this grabs the first five rows in that collection after it's been ordered, of course. So let's go ahead and run this now, and now you can see we just get the first five products. So Take allows you to take just a specific set of rows from the beginning of a collection. Let's go ahead and do the same thing with the method syntax right down here. So list = products.OrderBy, make sure you do you OrderBy first here because we want to take them after we've ordered, and then we do the ToList. So there is the same thing using the method syntax. And again, we can just try this out by changing our method in the program, and we see the exact same results.

Using the Range Operator with the Take() Method

With the Take method, you can also add the range operator. Now the range operator allows you to specify the start and an end of a range. For example, Take(5..8) gives us elements 6, 7, and 8. It does not include number 9. Now remember we're zero-based here. So 0, 1, 2, 3, 4, 5 would be the 6th element, but it does not go to the last one. It's up to that number. So for example, Take(..4) starts at the beginning, so it'd be element 0 up to element 3. Take (10..) goes from element 11 through the end of the collection. Take(^5..^2) starts at the 5th element from the end and goes to the 3rd element from the end. So lots of things you can do with this range operator. Let's take a look at using Take with range. Scroll back up to the TakeQuery method and copy the query you wrote using Take(5) and copy it down into the TakeRangeQuery. Now what we're going to do is let's do a (5..8). As we said, this will go from element 6, 7, and 8. And let's go ahead and grab this. Make sure it's filled in here. And let's run this. And as you can see, we get the three products that we were expecting. Now we can do the same thing, of course, with the method syntax. So if you go ahead and grab your method syntax that you wrote earlier, bring it down to the TakeRange method and do the 5..8. And once again, just try it out just to make sure that you've typed in everything correctly. And we should see the exact same results. Go back to your TakeRangeMethod, and let's explore some of the other options that we saw on the slide. For example (10..), if we run that, should start at element 11 and go through the end of the list. So we get 30 products returned. Or we can do (..4). So this grabs everything from element 0 up to that 3rd element, so 0, 1, 2, 3. We can also take things from the back. So you use the up caret, which

is Shift+ 6, 5, and then ^ maybe to the 2nd or the 3rd element from the end, right? So if we run that, we should see, again, we're getting things now from the end of the list. Lots of things you can do with the Take and the Range operator.

Conditionally Extract Data Using the TakeWhile() Method

Another method we can use called TakeWhile allows us to take elements while some condition is true. Let's take a look. Scroll down and find the TakeWhileQuery method and write the standard select with the orderby that we've done so far. And now what we'll do is right before the ToList, let's add on the TakeWhile. And inside of here, we're going to add a lambda expression, and we can say let's take all the ones starting with A, and we'll take those while if that condition is true. So it should return, in this case, if we take this, copy this over here as our method to run, we should end up with just a single value because there's only one product that starts with A in the whole collection, and there it is right there. We can do the same thing for the method syntax as well. If we go down to the next one, we can do our OrderBy and then add on the TakeWhile. And we could run this. We would get the exact same results.

Skip() Past Beginning Elements in a Collection

What goes along with the take is a Skip method. And this allows us to skip a specific amount of elements, and we can also even skip elements while some condition is true. Now, putting these two together can give us the way to do paging. So if you're doing, I want to show the first 10 records and then I want to show the second set of records, we can use a combination of take and skip. Or with the introduction of the range operator, you can now do everything was simply Take. But let's take a look at the Skip methods. Scroll down to the SkipQuery method and write again the typical select with the OrderBy and then add Skip. And I'm going to go ahead and just say skip 30. Let's go ahead and copy this one in and run this. And now what we should see is the last 10 items showing up because we skipped the first 30. Now we can also do the same thing using the method syntax, of course. So here we use the OrderBy and the .Skip, and we get the exact same results. Now there's another thing that I mentioned and that was paging. So if we wanted to, we could say let's skip the first five and then let's take the next five. So this would give you the paging capability. So let's go ahead and try this and see how this looks. So there we go. We get five products, but starting at that fifth element down in the list. Scroll down to the SkipWhileQuery method, and let's add on a SkipWhile. And again, we supply a lambda expression here. And we could skip while the first letter of the name property begins with A. So if we were to run this, it's going to give us back 39 items because at the very beginning, it skipped the first 1 because it skipped

while the name started with A. And if you remember, there's only one product that does that in the collection. And if you were to take this SkipWhile and go down to the SkipWhileMethod after you've written the typical select all ordering by the name, you can add that SkipWhile right onto the method syntax to get the exact same results again.

Get Distinct() Values from a Collection

Now sometimes we wish to get just a distinct value out of the list of products. For instance, we have a bunch of colors in there. We have a few whites, we have a few blacks, we have a few reds, etc. So what we might want to do is say I just want to see the distinct colors. I don't want them all duplicated. So let's take a look at how we can do that using LINQ. Locate the distinct query method, and let's write our typical from prod in products, and let's select prod.Color. So that'll give us just all the colors, and it'll bring that back as a list of string. And you can see on line 171, that's what I'm expecting. However, I'm going to apply the Distinct method to that list of colors, which will now just give me all of the ones that are unique. And then just let's go ahead and order these and convert it into our list. We take this method name, and let's make this one that we're going to run. There are our distinct list of colors. So this is nice, right? A lot of times you might be in a web application and you need a drop-down of colors out of the existing colors that are already there in your products, for example. This would be a way that you could accomplish that. Now, of course, you could do it with SQL as well, but here's a way to do it if you're getting it from some other data source. Go back and find the DistinctWhere method. And now, on line 190, you can write list = products.Select p p.Color. We want to grab just the color property. Apply the Distinct and the OrderBy just like we did in the query method and then the ToList to generate that list of string colors.

Extract Distinct Objects Using DistinctBy() Method

Now what if instead of getting that list of strings for each color, you wanted to get the actual product or one of the products that represented each of the distinct colors? You can do that with the DistinctBy method. Let's take a look. Locate the DistinctByQuery method and write a typical select syntax, so list = from prod in products select prod .ToList. So this selects everything. But what we want to do now is we want to apply the DistinctBy and supply a lambda expression that says what is the property that you want to get distinctly? So in this case, I'm specifying color just like we did before. Now what it's going to do is it's going to iterate over the collection, find the first product object that has a color and adds it to an internal collection, and then it throws away any others with that color until it finds the next color or one that is not in that list, and simply adds it again and again. So it iterates over this. Now I can then go ahead and order by, of course, the

color just to get all of the colors in order. But now what it's going to do is give me a list of products, as you can see on line 199. That's the type of object that I'm going to be returning from this method. So, let's go ahead and run this, and we'll now see we get still six items because we are getting all of those different colors, but we're getting a representative product object for each one of those. And once again, just to be complete, let's go down to the DistinctByMethod method, and let's write the method syntax for that DistinctBy. So now we can eliminate the Select method because we're going to use the lambda expression to choose which property in the DistinctBy, so list = products.DistinctBy prod arrow syntax prod.Color. Give me those that are distinct by color, order by the color, and then return the list of products.

Split Large Collections into Smaller Collections Using Chunk()

So in this module, we're looking at partitioning our data in some other format. And there's another way that we could take our large collection of 40 products and we could split that large collection into an array of smaller collections using the Chunk method. Let's take a look. Locate the ChunkQuery method in our SamplesViewModel class. And the first thing you'll notice is the return value. It is now a list of a Product array. Let's go ahead and write our query here. So from prod in products select prod, and then we're going to apply the chunk. Now what Chunk does is it says I'm going to take the first five from the collection, and I'm going to build an array out of those and put it into the first item of our list of product array. Then I'm going to go to the second five and build another array of those products. So set a breakpoint here right on the return list, and let's run this. And what we're going to end up with now, if we take a look, is we now end up with a collection of eight items where each item is an array of five product objects. Eight times 5 is 40, right? We had 40 products in our collection. And if we open each one of these, we can see that each one of them contains an array of five products. So it'll just keep doing this until it runs out. So the last item in our collection may be an array of only two or three depending on how many items you have left over after specifying your chunk size. This would be really great for doing paging if you had a smaller list like this. You could actually build the pages, keep them in memory somehow, and then, of course, display them as the user clicked through the different pages. And just to be complete, go down to the ChunkMethod method and write the method syntax here. So again, much simpler syntax when we're using the method syntax; whereas the query syntax, well, a little bit more readable but just more verbose. Again, choose whichever one you wish to use. This module was all about partitioning your data in a certain way, for example taking values from within collections or skipping values from the beginning of a collection or splitting a collection using the Chunk method. Now, these methods are typically used and are great for paging data. We also saw the Distinct and the DistinctBy methods that allows you to get unique values out of a

collection. Up next, determine the type of data contained within collections. I hope you'll join me for the next module.

Determine the Type of Data Contained within Collections

Introduction to the All() Method

Hello. Paul Sheriff here with Pluralsight. This module is Determine the Type of Data Contained within Collections. The goals for this module are to answer questions about a collection such as do all items meet a condition, do any items meet a condition, or does the collection contain a certain item? Now for normal int, decimal, strings, you can just compare that value against the values in the collection. But for a product class or a sales class, you'll need to implement an EqualityComparer class in order to do the comparisons. So what are some common uses of the All method? Well, the All method allows us to ask questions such as are all products price greater than their cost within a collection? Do all sales orders have a quantity greater than or equal to one? Do all customers have a zero balance? So this is what you might use the All method for. Let's take a look. The syntax for All is you apply the All method to some IEnumerable of T collection and you specify a predicate. This is then going to determine if all items within the collection match the condition. Here's an example, products.All prod arrow syntax prod.ListPrice greater than prod.StandardCost. So this will return a Boolean that says true or false, did everything in the collection meet this criteria?

Demo of the All() Method

Let's take a look at our first demo here of using the All method. Open up the SamplesViewModel in the start folder for this module, and you'll notice that we are going to be returning a Boolean value. So we're going to say value = from prod in products select prod. And then we're going to apply the All method here. And we're going to again pass in using the arrow syntax prod.ListPrice greater than prod.StandardCost. So we're trying to make sure that all of the products within our collection has a list price that's greater than the cost. So there you go. There is the sample query syntax there. If we then take this AllQuery and make sure that's the one we're calling from Program here, we run, and we can see the value comes back as true, which is exactly what we were hoping for. Go back here and let's write the same thing for the method syntax, products.All. And there we go. Same thing using the method syntax. Scroll down to the AllSalesQuery method, and let's write one using the sales orders now. So line 44, I create a list of SalesOrder objects using the GetSales method to retrieve that hardcoded collection. I'm then going to also have a Boolean value

I'm going to return. And we do our from sale in sales select sale, and then we'll do our All. And this time we'll do sale sale.OrderQuantity is greater than or equal to 1. We want to make sure that all items have an order quantity greater than 1. So, again, this will return a Boolean to us. If we run this one, again same thing. The value is true, so all of our orders are correct. Once again, come down here to the AllSales method and write the same code, sales.All sale arrow syntax sale.OrderQuantity is greater than or equal to 1. So there's the query syntax, as well as the method syntax.

Demo of the Any() Method

Let's now take a look at the Any method. Now the Any method is going to allow us to answer questions such as do any sales orders have a quantity greater than 10, for example? Do any sales orders have a total greater than 10,000? Do any customers have a credit balance? So these would be the types of questions you would answer using the Any method, the syntax. Again, you're going to apply the Any method to any IEnumerable of T collection passing in a predicate, and it's going to determine then if any items in the collection match that condition. So for example, sales.Any, passing in sale arrow syntax sale .LineTotal greater than 10,000. Let's take a look at a demo of using the Any method. Locate the AnyQuery method. And again, we're doing the same thing with the sales orders where we're getting the sales as a collection and we're going to return a Boolean again. So if we write the query syntax for this, it would be from sale in sales select sale. And now we're going to use the Any method sale.LineTotal greater than 10,000. So this says does any sale within our sales order collection contain a line total that's greater than 10,000? Let's go ahead and copy this over to our Program and run. Now, for this particular collection of sales, there is no line total that has a value that's greater than 10,000. Let's go ahead and see the method syntax for this, value = sales.Any sale arrow syntax sale.LineTotal greater than 10,000.

Demo of Contains() Using Integers

Let's now take a look at the Contains method. Now this searches a collection to see if a value exists, very similar to the Contains method of a string. For simple data type collections such as int, decimal, string, things like that, what we're going to do is check to see if the value in the collection is equal to the value you are searching for. Let's take a look at what happens when we use Contains using an integer list. Locate the ContainsQuery. On line 110, you see I'm creating an integer list. So I've got numbers equal to a new integer list of the numbers 1, 2, 3, 4, and 5. Let's go ahead and write our typical query. From num in numbers, select num so that gives me the list of all the numbers, and I want to see if it contains a 3. So we're just going to check to see is a 3 contained within that list. Now as you

can kind of guess here, that's going to give us a true value. Go ahead and try it out. And you see that, yes, that is true. It does exist in there. Very simple when we're checking against numbers or strings or anything like that. You simply pass in the value. And we can do the same thing for the method syntax, value = numbers.Contains the number 3.

Demo of Contains() Using Comparer Class

So searching for simple data types is very simple with Contains. But when you're doing Contains with objects, the way this works is that, by default, it's going to just simply compare object references. And you're probably going to want to be able to look at a value in one or more properties of the object. So to do that, you're going to need to create an EqualityComparer class, which is what we're going to show you how to do. But the way this works is you create a class that inherits from the EqualityComparer. So in this case, I'm saying I'm going to be doing a comparison on a product object. You have to override the equals method that's a part of the EqualityComparer class, and you're going to pass in two of those objects, in this case product. So we're getting product 1 and product 2. You then write an expression, which is what you want to compare on. So if you're just comparing the product ID, you would just do that. If you're comparing the product name, you would add the product name in here. Whatever you need to perform to do the matching belongs here in the equals. You also have to override a GetHashCode method, and this has to return a unique ID for every single object. So ProductID in my product scenario is a unique value. If I didn't have that, I would simply include all properties from the product object. All right, let's take a look at Contains using a comparer class. Expand the LINQSamples.Common project and then expand the ComparerClasses folder and locate and open the ProductIdComparer.cs file. Inside of here, you find the public class ProductIdComparer that inherits from EqualityComparer product. This class performs the functionality I described in the slide. Now go over to the SamplesViewModel. Locate the ContainsComparerQuery method. We're still getting our list of products. But now on line 143, we create an instance of the ProductIdComparer class. Now we can write our query from prod in products select prod. Use the Contains method. And what we're going to do is the first parameter is what we want to compare. Well, we need a product object since we're going to compare a product to a product. So we'll create a new one here and assign the ProductID equal to some value that we want to search for. Remember, that's what I'm comparing. And we then, to the second parameter, which is an optional parameter to Contains, we pass in our ProductIdComparer. Now, let's go ahead and set a breakpoint over here so we can see what happens when we run this method. When it stops here, we can hover over X and we see the value is 680. We see the y is the value we are searching for, the 744. Go ahead and hit Continue. We come in again. This one now has the next item in the collection, and it's now comparing it against that 744

again. So this just continues until we either find it or it does not find it, and it will return back true or false based on whether or not we found the product we were searching for. Scroll down and locate the ContainsComparerMethod, and we can write the same Contains, creating a new product. Set the ProductID equal to 744 and to that second parameter pass your ProductIdComparer. In this module, we took a look at the All method to check if all items within a collection match a certain condition. We also saw the Any, which checks to see if any items match a specific condition. We also used Contains, and we used Contains with a comparer class to make it easy to search object property values, not just comparing object references. And don't worry. We're going to see more examples of comparer classes coming up later in this course. But you will need to build a comparer class for each type of search you do wish to perform on each of your objects. So, depending on what you're searching for, you may end up with a multitude of comparer classes. Up next, determine the differences between two collections. I hope you'll join me for the next module.

Determine Differences between Two Collections

Using SequenceEqual() with Integer Collections

Hello. Paul Sheriff with Pluralsight. This module is Determine Differences Between Two Collections. The goals for this particular module are to show working with two collections. We're going to find out are the collections equal? We're going to find out if there's values in one and not the other. And we'll find out the values that are in common between the two collections. We're going to explore several different LINQ methods to accomplish all this functionality. Let's start out looking at the SequenceEqual method. Now this method compares two collections for equality. If you're using simple data types such as integer and decimal, it simply checks the individual values. Objects, however, it's going to check to see if they are the same reference to the same object. But we can also use a comparer class to check values in the properties just like I showed you in the last module. There's a lot of uses that you can use SequenceEqual for. For example, you could read lines from two text files and see if they are the same. You could read a color field from two different tables and see if the values are equal. You could read customer data from two different tables and use a comparer class to see if they are equal. You could read columns from two tables in two different databases to see what is different. Think a QA and a production database, for example. Let's take a look at our first demo of using SequenceEqual with integers. Open up the project in the start folder for this module. Open up SamplesViewModel class and find SequenceEqualIntegersQuery method. And you'll see on line 11, I have a list of integers that start with 5 and then 2, 3, 4, 5. And then on line 13, I have another

list of integers that go 1, 2, 3, 4, 5. So what we're going to do is we're going to write the syntax that you see here on line 16, so you can start typing this in, value equals from num in list1 select num. So that gives us the list of all of those first integers. And then we invoke the SequenceEqual method, passing it list2. I want you to realize that I'm using the query syntax here just to be complete and consistent in everything that I'm teaching. But realize that list1, you could just substitute that there since all we're doing is selecting everything in list1. So we could just say list1.SequenceEqual, basically the method syntax. As I mentioned before, the query syntax and the method syntax basically do the same thing, and everything really breaks down into the method syntax anyway. But I wanted to at least point that out that doing the from with the select is really unnecessary. But I want to do it just to stay consistent throughout this course. So basically, it's going to return a Boolean that says are all the values equal. So it just does a direct one-to-one comparison between all the values in these two lists. So make sure that this method is the one in Program.cs, which it is. And let's go ahead and run this, and we can see that the value is false. And the reason why is because it just simply goes and checks the first element against the first element in the other list and the second one and the second one until it finds one that is not valid and then it bails out and it returns a false. If we were to change this one here to a 1 so that they are exactly the same, now you can see that we get the value back of true. Now scroll down to the SequenceEqualIntegersMethod method. Has the same two lists, but now you can write the method syntax here, value = list1.SequenceEqual and then pass in list2.

Using SequenceEqual() with Object Collections

So for any simple data type such as integer or decimal, that's all you have to do. If you want to use objects, we're going to have to do a couple of things. First, let's take a look at using SequenceEqual using object references. Locate the SequenceEqualObjectsQuery method. Here now you're going to see two different lists. But what I'm doing is creating a list of products, and I'm just hardcoding a couple here. You can see lines 50 to 53, I create two new products, Product 1. And the name is Product 1 and ProductID 2 Name equals Product 2. Then on line 55, I create another list of products, but they look exactly the same. So now, if we were to write the code that we're seen here on line 62 and 63, value = from prod in list1 select prod and then SequenceEqual list2. Let's go ahead and run this. After you've typed that in, go ahead and copy that method name over to Program and run. Now you can see, I'm going to get a value back that is false. Why? Because remember when you're using objects it's checking to see if each list points to the same object reference. So for example, what we could do is we could say list2 is equal to list1. That will then make them exactly equal. Because now they're each pointing to the same product object. If we run this again, you now see that we get a value of true. So with objects, by default, if you're comparing objects, it's checking object references. And of course, just for

completeness here, you can also write the same method syntax that you see here on line 94, value = list1.SequenceEqual(list2).

Using SequenceEqual() with Comparer Class

Now instead of checking object references, most likely you're going to check to see if the product objects themselves using the properties is all the same. Well, for that, we need to use a comparer class just like I showed you in the last module. So let's take a look at using SequenceEqual using a comparer class. Expand the LINQSamples.Common folder and then the ComparerClasses folder. And now, bring up ProductComparer.cs. This is just like what we wrote before, but a little bit more involved. We're still going to inherit from the equality comparer. We're going to override the Equals, passing a product x and a product y. But what we're going to do this time is we're going to compare each property within the Product class to see if they are all equal. So we simply do return x.ProductID equals y.ProductID and x.Name equals y.Name and so on through the list of properties. We also have to write our GetHashCode method. We override that one. So on this one, what I'm going to do is I'm going to take each property, and I'm just going to simply concatenate those all together, put them into a variable called value, and then return value.GetHashCode. So once we have this ProductComparer, locate in the SamplesViewModel SequenceEqualUsingComparerQuery method. Now you can see on line 106, I create a ProductComparer. Pc equals a new ProductComparer. And then list1, I'm going to do a ProductRepository.GetAll, and then list2 equals ProductRepository.GetAll. So right now, both those are the same. They're not the same objects. They're different objects as far as object references go, but they have the same data. So I now come down here and I write that same query. You see on line 116 and 117 where from prod in list1 select prod.SequenceEqual list2, and now we pass a second parameter to sequence, which is that ProductComparer. If we now make this the method to run, we can see that, yes, indeed, based on all of the properties for each item in each collection, they are equal. But what we could do is I could simply remove maybe a single item from one of the lists. And if we do that and then we run, now we're going to get back a value of false because they are no longer equal. And again, the same thing down in the SequenceEqualUsingComparerMethod method, we write the method syntax that you see here on line 139.

Using Except() with Integer Collections

Let's now take a look at another LINQ method, and this one is called Except. This allows you to find all values that are in one list but not the other, and then it returns the list of the exceptions. Remember, SequenceEqual gave us a Boolean value. This one's going to tell us what is different between the two lists. So we

could use simple data types. It's going to check the values. Or if we're doing object data types, it's going to check the references just like we're used to. And of course, we can always use a comparer class to check values in properties. So all of these LINQ methods are starting to work the same. We're going to do object data type references, or you use the comparer class. You're going to see this same pattern over and over again. So, what kind of uses could we do with the Except? Well, we could find those products that don't have sales, for instance. We could find those customers that have not ordered a specific product. We could find out what data differences exist between QA and production databases. You could go read the schema and use the Except method to figure out what's missing. What column is missing, what stored procedure is missing, etc. All right, let's take a look at our first demo here, and that will be doing Except using integers. Locate the method ExceptIntegersQuery. On lines 152 and 154, you're going to see two different lists. The first one, 5, 2, 3, 4, 5, and the second one, 3, 4, 5. So obviously, there are differences. So what I can do now is I can write on lines 157 and 158 list equals. So we're getting back a list of integers. From num in list1 select num, so that gives us all of the integers in that first list. And then we apply the Except method passing in list2, and this will give us back our list of integers that are not in common. So let's go ahead and take a look at this, ExceptIntegersQuery. Let's run. And now, we can see the only one that was not in common was number 2. Go back here and look. See 3, 4, and 5 are all in common; 2 is the exception. We can also write the same thing using the method syntax as you see here on line 176.

Find Products That Do Not Have Sales Using Except()

Now that you've seen the simple example, now let's take a look at finding products that do not have sales. Expand the LINQSamples.DataLayer project, expand the RepositoryClasses folder, and take a look at the ProductRepository. We've seen this before where I've got each of the products in here with a ProductID. And if you take a look at the SalesOrderRepository, you'll see the sales orders. It's got a SalesOrderID. We have an OrderQty and then a ProductID. And you can see that as we move down here, I've got for SalesOrderID 71774, I've got the first one has a ProductID of 680. They're also ordering 10 of ProductID 712. And that's the total order of those two line items. So that's what we're looking for. So what I want to do is I want to find out out of all the products in this big long list of products, which ones don't have sales? Back in SamplesViewModel, locate the ExceptProductSalesQuery. What we're going to do is return a list of integers that will be a list of ProductIDs that do not have sales. So first, grab all the products. Then, grab all the sales. On line 208, write list equals from prod in products select prod.ProductID. So that gives us the list of ProductIDs from the Products table and then do a .Except. And now, from sale in sales select sale.ProductID. That gives us all of the ProductIDs in sales, and then the Except method will give us just the difference. So let's go ahead and try this

out. Let's grab our method name, paste that into Program, and now do Run, Start Debugging. And what we'll see here is 19 products that do not have any sales. We can also go back and we can come down and look at the same thing expressed as the method syntax. So find ExceptProductSales method. We do the same thing. Grab all the products. Grab all the sales. And now on line 227, you'll write list equals products.Select prod arrow syntax prod.ProductID .Except sales.Select sale arrow syntax sale.ProductID.

Using Except() with Comparer Class

And just to make sure I'm showing you everything possible, I want to show you using Except using a comparer class. Locate the method ExceptUsingComparerQuery, and you'll see on line 222 ProductComparer pc = new. So I'm creating our ProductComparer. That's the one that tests all of the properties. On line 224, get a list of product data. And then on line 226, get another list of product data. So think about maybe you're getting the first list from your QA database and your second list from your production database because you want to see if you've got the right data contained in there. And then just to simulate like they might be different, on line 230, I'm going to write list2.RemoveAll, I'm going to remove all using a lambda expression here, prod prod.Color is equal to black. So the RemoveAll method will allow us to delete values from a list. Now, when I write my query syntax here on lines 233 and 234, list = from prod in list1 select prod.Except list2 and then we use our product comparer in that second parameter. What we get back now is a list of all the products that are different. So let's go ahead and put that into there and run this. And we see we've got 10 products, and it's all the ones that are black because those are the ones that are the exceptions, which is exactly what we are expecting to get. Now this was just a little bit of a contrived example, of course, but you can see where if you are reading this from two separate databases, we might get different values. I was just simulating that here. And of course, you can use the method syntax that you see down here on line 257, list = list1.Except list2 comma our product comparer.

Using the ExceptBy() Method

Starting with .NET 6, Microsoft has introduced ExceptBy method. This allows you to find all values in one list but not the other, just like the Except. It returns the list of objects that are different, but here's the kicker. You don't need a comparer class to use this, which is kind of neat. So let's take a look at using the ExceptBy using a key expression. I'm going to show you two examples of ExceptBy. Let's start out the first one, find the ExceptByQuery method. What I'm doing in this one is on line 271 is I'm getting all the products. On line 274, I'm creating a list of strings. It's called colors, and it's a new list that simply has red and black. These

are the ones I want to exclude from the lists that I get back. So what I'm going to do here is I'm going to say list = from prod in products select prod, so we're getting all the products, ExceptBy and then you pass in that list of colors. And you say okay, the second parameter that I'm passing to this now is the key expression, which in this case is the color property on the product. And what it's doing is saying okay, let's get everything except that color red or black. Let's take a look. Copy this over to Program, and we will run this. And we get four products back. We get a color white, a color blue, a color multi, and a color silver. Why are there only four products returned? We know there are more white products, for example. Since the key expression is color, it is only showing you those products with the colors other than red or black. And of course, we can also use the method syntax as you can see here on line 298.

Find Products That Do Not Have Sales Using ExceptBy()

I've already shown you how to use the Except method to find products that do not have sales. Now let's take a look at how to use the ExceptBy method to perform the same function. Locate the ExceptByProductSalesQuery method. This one is going to return a list of products because that's what the ExceptBy will do. It returns a list of products, not just the IDs like we saw with the Except. So we're going to get all products. We're going to get all sales. Then let's start writing our query syntax, list = from prod in products select prod, there's our type that's going to be coming back, .ExceptBy. Now, when we used ExceptBy before. The default for the key value that we're keen on, the key expression, is string. If you want to change that, then you have to pass two parameters to this generic method. One is the type of things that are going to be in this collection, and the second is the data type of the key expression. We're going to be using ProductID, which is an int. To the ExceptBy, you pass in from sale in sales select sale.ProductID comma. The second parameter then is the expression that gives us the ProductID from the products collection. It compares those two, and then it says these are the ones to give back as the products that come back into our list. Let's go ahead and take this method name. Paste it in here. Let's run. And just like we saw for the Except, we get back 19 products. But as you're seeing, they are now the full product objects that are coming back. All right, let's go ahead and write the same thing using the method syntax. So find the ExceptByProductSalesMethod method. Get all the products, get all the sales, write the method syntax, list = products.ExceptBy your two parameters, product comma int and now sales.Select, all the product IDs, comma prod.ProductID. Get them all from products, compare the two, and bring back the list of products where those product IDs are the exception.

Using Intersect() with Integer Collections

Let's now take a look at the Intersect method of LINQ, and this one allows you to find all values that are in common between two lists. So basically, it's the opposite of Except. So for simple data types, int, decimal, etc., it checks the values. If you have object data types, it checks references. And of course, you can use a comparer class as well to check the values in properties. So let's take a look at our first demo, which will be the simple one. And we'll do an Intersect using integers. Locate the IntersectIntegersQuery method. This should look very similar. I'm using the same lists that we did for the Except where the first list has 5, 2, 3, 4, 5, and the second list has 3, 4, 5. Now I'm going to do my list = from num in list1 select num, so I get all of the integers in list1, and I'm going to Intersect that with list2, which now will give us back all of the numbers that are in common between those two lists. Let's go ahead and copy this one over, and we should get back now our three items, which are the 5, 3, and the 4. And we can also write the same thing using the method syntax that you see here on line 372.

Find Products That Have Sales Using Intersect()

Let's take a look at a real-world example of where you would want to find products that have sales. And for that, we're going to use the Intersect method. Locate the IntersectProductSalesQuery method. We're going to return a list of integers. It'll be the list of product IDs that are in common. So we're now going to find all products that have sales. We get the list of all products; we get the list of all sales. Now let's write our query syntax, list = from prod in products, select prod.ProductID, apply the Intersect method, and we're going to intersect that with all of the product IDs from sales by doing from sale in sales select sale.ProductID. This now returns all the ones that are in common. Let's go ahead and copy this and paste this into our Program.cs and run. We now get a list of 21 products that all have sales. Now we can write the same thing using the method syntax. Locate the IntersectProductSalesMethod method. Still grab all the products, still grab all the sales. Write the syntax list = products.Select prod arrow syntax prod.ProductID .Intersect with the sales.Select sale arrow syntax sale.ProductID. Again, connects up the two and gives us those that are in common. Thus we find all products that have sales.

Using Intersect() with Comparer Class

And let's finish this out by showing you an intersect using a comparer class. Locate the IntersectUsingComparerQuery. On line 418, we create a new product comparer. And then on lines 420 and 422, we build two different lists of products. Now again, think about you're maybe getting the products from one database and the products from another database, and what we want to do is find which ones are in common. I'm going to simulate that by going into list1 and removing all where the product color is black. Then I'm going to go into

list2 and remove all the products that are red. So now, when I write my query syntax, list = from prod in list1 select prod, I get back all that list of products. I intersect that with list2 using my product comparer, and it will now give me back the list of which ones are in common between those two separate lists. And here you can see we get 12 products back. If we scroll up, we see it's the multi, the blue, the white, and the silver, no reds and no blacks. And we go down here to our same method by using the method syntax, and you can see that here on line 455.

Using the IntersectBy() Method

Starting with .NET 6, Microsoft has introduced the IntersectBy method. This allows you to find all values in common between two lists. It returns the list of objects that are the same, but you don't need a comparer class. So, let's take a look at using IntersectBy and using our key expression. Locate the IntersectByQuery method. We're going to create a list of products that's going to be returned, and let's go ahead and grab all of our products. Next, on line 470, I'm going to do exactly what I did before. I'm going to create a list of strings that is colors, and I'm going to create a new list here with red and black. So I'm going to write my query syntax list = from prod in products select prod, so I grab all of the products, IntersectBy that colors list and using the key expression p.Color. So it compares and finds all of those that match up. And if we now run this, when we intersect those two lists, we get the distinct values of black and red. So it's the opposite of the ExceptBy. And we can write the same thing using the method syntax that you see on line 492.

Find Products That Have Sales Using IntersectBy()

Let's now use the IntersectBy to find products that have sales. Locate the IntersectByProductSalesQuery method. You're going to get all products and get all sales. And now let's write the query syntax, list = from prod in products select prod.IntersectBy. Now, just like we did with the ExceptBy, the default data type for the key expression is string. If you want to change to anything else, you must pass in two parameters to the generic method IntersectBy. The first one is the type of data in the collection; the second one is the key expression data type. So we're going to do ProductID so it's an int. We then pass to the IntersectBy. The first parameter is from sale in sales select sale.ProductID, get that list of product IDs. The second parameter is prod.ProductID. So you tell it the key expression to get out of the product collection. It matches those up and returns the list of products. Let's go ahead and copy this into our Program.cs and run it. And we get back the 21 products just like we saw before. But this time, it's now the complete list of the products themselves. Let's go ahead and close this, and let's go to the IntersectByProductSales method. Get the products, get the sales, and now let's write the same thing using the method syntax, list = products.IntersectBy,

passing in product and int. To the first parameter, we pass sales.Select s.ProductID. Second parameter is p.ProductID. Matches them up and returns the list of products. In this module, we saw how easy it is to compare the values between two collections. Now with simple data types, you're just checking the values. It's automatic. With object data types, you're going to need a comparer class, but you do have the option of using the By methods which eliminate the need for the comparer class. Up next, concatenate collections together using Union and Concat. I hope you'll join me for the next module.

Concatenate Collections Together Using Union and Concat

Using Union() with Integer Collections

Hello. Paul Sheriff with Pluralsight. This module is Concatenate Collections Together Using Union and Concat Methods. The goals for this module, we want to combine two collections together, and we're going to do this with three different methods. First one is Union. Now Union will not give us any duplicates that are between the two lists. UnionBy will also not give us any duplicates between the two lists. And Concat, and Concat, as you would expect, gives us the duplicates. Let's just dive right into our first demo and show doing a union using integers. Locate the UnionIntegersQuery method. And we're going to create a list1 that has the integers 5, 2, 3, 4, 5 and list2, which has a list of integers 1, 2, 3, 4, and 5. What we're going to do on line 19 is we're going to say list = from num in list1 select num. So we're getting that list of all of the items from the first list, and we're unioning it with list2. So we're putting them together. And then I'm going to go ahead and order them. I want to order them by the numbers so we can see what happens. Let's go ahead and make sure this method is the one that we're going to call here in Program and go ahead and run this. And you can now see what happens when I ordered them. I get them all in the nice order, 1, 2, 3, 4, and 5. So what it did when we unioned is it eliminated the duplicates. And we can write the same thing in the method syntax, as you can see here, on line 41. So same lists, but now we're just doing list1.Union list2. And then of course, you can add on the OrderBy if you want.

Using Union() with Comparer Class

Now that we've seen the simple integer approach, let's do the same thing using our product objects, and let's union using the comparer class that we also wrote earlier. Locate the UnionQuery method. On line 56, you can see we're creating a new instance of the ProductComparer. Remember that's the one that compares all of the properties in the product object. We then create list1 by getting all of

the product data, and we create list2 by getting all the product data. We now write our query syntax, list = from prod in list1 select prod.Union list2 using our product comparer, our comparer class. And then I'm going to go ahead and order those by name again. Let's go ahead and grab this method. Make sure that's the one we're calling. And you see we get only 40 products. So that means it eliminated the duplicates using that product comparer. And we can also write the same thing using the method syntax that you see here on line 86.

Using the UnionBy() Method

Now just like we had an ExceptBy and an IntersectBy, Microsoft has also introduced UnionBy starting with .NET 6. So let's take a look at doing a UnionBy using a key expression. Locate the UnionByQuery method. Now in this one, I'm going to do on line 100 the same that I did before. I'm going to create a list of products and I'm going to load all the products in list2 as well. And now, on line 105, we do our from prod in list1 select prod. We're going to use the UnionBy. We're going to pass in list2, and then the second parameter is our key expression. So we're saying I want to actually do the union in by color. And I'm going to order by the product name and then convert that to a list, of course. Now, just like the ExceptBy, this one is also going to do the Distinct with that color. Because it's a string value, it's going to say, hey, I need to get a distinct value here. So we'll go ahead and hook this up and we get six products because that's one for each of the various colors, multi, silver, black, red, white, and blue. And we can also do the same thing using the method syntax that you see here on line 126.

Using Concat with Two Integer Collections

Let's take a look at the Concat method now that helps us join two integer lists together. But this time, we'll include duplicates. Locate the ConcatIntegersQuery. You see our two lists again where the list1 has 5, 2, 3, 4, and 5, and list2 has the 1, 2, 3, 4, 5. When we write the query syntax, it's going to be list = from num in list1 select num, so it's getting all of those numbers, concatenate on list2. So this is doing a direct concatenation. Now let's do this. Before I do the OrderBy, let's just run it this way. And let's run this because this will show us what we get when we are literally doing a concatenation of the two lists together. If I add the OrderBy back in, then when I run this, now you can see there was only one. There were two 2s, three 3s, four 4s, and three 5s. And of course, we can do the same thing using the method syntax as shown on lines 168 and 169.

Using Concat with Two Product Collections

And for our last demo, let's use Concat with two product collections. Locate the ConcatQuery method. Now usually, at this point, I would be showing you a product comparer. But think about what Concat is really doing. It's really simply adding one list to another. So there's no reason for a product comparer. Why do we want to compare something if it's not going to check for duplicates anyway? So, we create our two lists. We write our query syntax on lines 189 through 191 where we get all of the list1, concatenate with list2, order by the product name. Let's run this, and we should see we get 80 products. And if we go all the way to the top, you can see that each one is duplicated. Each product is duplicated because I had two separate lists. We can also do the same thing using the method syntax that you see here online 211. This module was all about combining lists. Now we can use Union and UnionBy to combine lists, and these will check for duplicates. So you're going to make sure that you end up with no dups in the final list. Or you can use Concat, which is really just like adding one collection to another collection, and there's absolutely no duplicate checking. Up next, use the join clause to combine two collections. I hope you'll join me for the next module.

Use the Join Clause to Combine Two Collections

Performing an Inner Join

Hello. Paul Sheriff with Pluralsight. This module is Use the Join Clause to Combine Two Collections. The goals for this module are to perform an equijoin, also known as an inner join between two or three collections, create a one-to-many using a group join, and we're going to show you how to simulate a left outer join even though LINQ doesn't explicitly support it. An equijoin or also known as an inner join in SQL means that you have two or more collections as needed. And within the object, there's at least one property in each that share some sort of value that can be equal, and that's how we do the joining together. So let's take a look at a demo of doing an inner join. Open up the start folder and open the LINQSamples project. Open SamplesViewModel and locate the InnerJoinQuery method. Now you notice this one is going to return a list of ProductOrder objects. Now the ProductOrder is a class that I created. Go down underneath DataLayer, expand CompositeClasses, and you'll find ProductOrder. Now ProductOrder is a combination of fields from the product object and from the sales object. So you can see I've created a product ID, name, color, standard cost, list price, and size all from Product. Then, down here, on lines 13 through 16, I've grabbed some of the properties from the SalesOrder object. I have created all of these as nullable properties. And the reason why is because later on when we do a left outer join, you're going to see how these could

potentially equate to a null so we need to be able to support that. All right, let's go back to the SamplesViewModel. I'm going to create my list of ProductOrder called list on line 12. And then on lines 14 and 16, we'll grab all the products and we'll grab all the sales. Now we can start writing our query syntax, list = from prod in products, let's use the new join clause that I just talked about, join sale in sales on, so just like you would do in SQL, prod.ProductID. There's the property that's going to equal now the ProductID in the sale collection. So it's going to join those up. And then, for each time through that it finds one, we're going to select a new ProductOrder. So we're creating a new ProductOrder object, and we're going to do ProductID = prod.ProductID, name = prod.Name. And we keep going down and filling in each one of the properties from either the product or the SalesOrder object. We finish by doing an order by p.Name, and then we have our list now of product orders. Let's make sure in Program.cs that this is the one we're going to run. Go ahead and start. And what we get back now is a list of all products on all orders. So the AWC Logo Cap, that's on order ID 71774, and it's also on order ID 7182. And if you could just keep going down, you'll find for each one of these like HL Road Frame Black. ID, the ProductID is 680. It's also on 7174 and 71776. And there you go. There's your join of the products and the sales together to find those products that are on sales orders. We can also express the same thing using the method syntax. Under InnerJoinMethod, you would come down and you would grab the products and grab the sales. You would then write the method syntax, list = products.Join. Now we're going to use this join method passing in what we want to join to the sales collection. And then the second parameter to the join is the key expression prod.ProductID. The third parameter is the sale.ProductID. So those are the two fields we're going to match on. It's then going to pass to this anonymous function prod and sale, and it's going to build a new ProductOrder each time through and, again, filling in the appropriate properties from the appropriate objects, product and sale. We finish by doing an OrderBy the name.

Using a Two-field Inner Join

Now we just joined on one single field to the other single field in the other collection. But what if you wanted to join two or more fields together? Well, the syntax is just a little bit different, so let's take a look at that now. Locate the InnerJoinTwoFieldsQuery method. We're going to again load up products and sales. And then for our query syntax, list = from prod in products join sale in sales. So that part's the same. But the on is, instead of a single field, we're now going to create an anonymous class that we list what we're trying to join on. So I want to do prod.product ID comma Qty = 6. So we have now two fields, equals, new, sale.ProductID, so we're just lining these two classes up positionally, comma Qty = sale.OrderQuantity. So basically, we're saying match up the product IDs. Match up where the quantity 6 is equal to the sale.OrderQuantity. Where those conditions meet, grab those and create our new product order, filling in

the appropriate properties just like I showed you before. Let's copy this over to our Program and run this to see the results. What we end up with now is only two products because they're the only ones where the product IDs match and the order quantity is equal to 6. Let's now go back over here and take a look at how we would do the same thing using the method syntax. So locate the InnerJoinTwoFields method, load up your products and your sales, and let's write the method syntax. So list = products.Join, pass to that method sales, and now we list our two key expressions, which in this case are those new anonymous classes that I'm assigning to prod and sale, respectively. So prod.ProductID comma Qty equals 6 new sale.Product ID comma Qty equals sale.OrderQuantity. Pass those two new variables, prod and sale, into the anonymous function, and then we create our new product order there, passing in all of the properties from the prod and the sale into our product order object.

Using the 'into' Keyword

Let's take a look at doing a join with an Into. So we're going to use these two keywords here to create a new object with a sales collection for each product. So what we're going to do is list each product. And then underneath that, so to speak, we're going to create the sales that go along with that product. Now this query syntax uses both the join and the into keywords. The method syntax uses another method called GroupJoin. What we're going to accomplish with this join into is we're going to create an object that has a SalesOrderId and then has a list of product objects that correspond to those sales orders. Let's first take a look at the query syntax, which uses the into keyword. Locate the JoinIntoQuery method. Create our list of ProductSales objects this time. Now the ProductSales is again one of these new classes that I created down here underneath that CompositeClasses folder in the LINQSamples.DataLayer project. So here you can see ProductSales. What it is is it's a product object and then a public list of SalesOrder objects. So we have a product, and we have a sales property. Go back here to the SamplesViewModel, and now let's write our query syntax, list = from prod in products orderby the Product ID. So I'm going to actually make sure that everything's in product ID order. We're going to join sale and sales on the ProductID equals the sale.ProductID. And here's the thing. We're going to use the into keyword newSales. What that does is that takes that sales collection for each product and puts it into this variable. So I can now create a new instance of my new ProductSales. And I can assign prod to the product property within there. And to the sales property, I can assign the new sales.OrderBy the sales.SalesOrderId. And there we go. We now have a list of products with their corresponding sales as a collection. Let's grab this method name, put it into Program, and let's run this. So let's go all the way to the top, and now we can see the first one, HL Road Frame Black. The product ID is 680. And then it shows you the orders for that product. There's one on order ID 71774, and there's one on order ID 71776. We then go to the next one, HL Road Frame, so product ID 706.

There are no sales for this particular product. We then go down to the next one, Sport 100 Helmet, product ID of 707. It's on one order, which is the 71780, and so on. Now, this is really neat. This is a very popular thing, a very common thing that you would do in many applications where you have a one-to-many relationship.

Using the GroupJoin() Method

Next, let's take a look at using the method syntax which uses the GroupJoin method. Let's scroll down now and locate the JoinInto method, and let's learn how to write this same thing as a method syntax. Now this is different. Normally, it's been a pretty much a one-to-one correspondence. But when you're doing the grouping, it's a little different. We grab our products. We grab our sales. And then on line 190, list = products.OrderBy ProductID .GroupJoin. First parameter is the sales. The second parameter is the first key expression that we want to join. The third is the next key expression, which is on the sale.ProductID. Then, what it does is to the anonymous function passes the product, and then it passes the list of sales for this product. We then create our new product sales, so product = our prod, and the sales property = newSales.OrderBy our SalesOrderID. So just a little different than the query syntax, but it accomplishes the exact same results.

Simulating a Left Outer Join (Query Syntax)

Let's now look at a left outer join, and the query syntax and the method syntax again are a little bit different. So we're going to focus first on the query syntax. Now what we're going to do is we're going to use an inner join using an into, but we're going to add on a second from statement. Now a null object may be returned for the right collection that we get back from the from. So we will use DefaultIfEmpty method to give us an empty object if there is nothing that matches in that right collection. Let's take a look at our first demo of a left outer join. Locate the LeftOuterJoinQuery method. We're going to be returning a list of ProductOrder, so that's that same one we used before. We're going to get all products. We're going to get all sales. Let's write the query syntax here, list = from prod in products. So that's our left side. And if you remember in SQL, when you say a left outer join, it says give me all the data from the left side and only those that correspond on the right. So we're going to do that by joining sale in sales on prod.Product ID = sale.Product ID. We're putting it into newSales. So that means give me all the sales for that ProductID. Now we're going to add on this extra from, and we're going to say from sale in newSales. Now think about this. If the product has sales, that newSales has data. So when we create a new ProductOrder, we get the product data and we get the sales data. But think about this. Let's say the product has no sales. That's where the newSales.DefaultIfEmpty comes in, which means there are no sales so the DefaultIfEmpty means give me back a null value into that sale property. This sale

property here only corresponds to the join. This sale property is another one that's unique, and it's the one that we use when building the new ProductOrder. And you'll notice what I did down here on lines 229 through 232. When I used the sale variable, I'm using a question mark after it. That's the null conditional operator that says if the value is null, then put a null in the SalesOrderID. If the value is null, put a null in the OrderQuantity, etc., etc. But if it's not null, go ahead and grab the SalesOrderID. In this way, I now have the opportunity to say give me the sales data if it's there. Otherwise, just fill these with a null value. Now you see why on lines 13, 14, 15, and 16, I declared each of these variables as a nullable type because I wanted the setup for using a left outer join. So let's go ahead and copy this and paste this in here if it's not already there, and let's run this and we'll see the results. Let's go all the way back to the top, and we can see that the first couple, yes, they have sales. But that third one, the HL Mountain Frame Black 42, the ID 743, does not have any sales. Now I did that by using a ToString method to spit that out. But let's do this. Let's take a look at what this looks like if we set a breakpoint. Go back over to the SamplesViewModel, and let's set a breakpoint right down here where we return the list. And let's run this again. And now when it stops, we'll hover over list. If you remember, it was that third one down, the HL Mountain Frame. If we expand this, we can see that the LineTotal is now null. The OrderQuantity is null. The SalesOrderID is null, and the UnitPrice is null. So that sales data is null because there were no sales for this one product. If we go up and we look here, we can see that those values are filled in because there was sales data for that particular product. So there we have a left outer join using the query syntax in LINQ.

Simulating a Left Outer Join (Method Syntax)

Let's now take a look at the left outer join using the method syntax. Now for this, we're actually going to use the SelectMany method to select the right collection, which, in this case, would be our sales. We're going to use the Where method to filter what is selected in the right collection. And of course, we use the DefaultIfEmpty method for that right collection so that we get back a null for the sales. All right, let's take a look at using the left outer join using method syntax. Locate the LeftOuterJoin method. We're going to do the same thing. Get the products, get the sales, and then we're going to write the method syntax. For the method syntax, we're going to do list = products.SelectMany. Now what happens is the product object gets passed in to the right side of the arrow function that you see here where we're going to do sales.Where. And then what we can do is we can say, okay, take each sale, try to find where the product ID in the sale is equal to the current product's ProductID. Now if it finds it, great. It then passes each of those, the prod and the sell, to the anonymous function you see as the second parameter to the SelectMany, and it creates the product order just like we did before. Now, if that sales.Where does not return anything because there is no sale with that ProductID, that's where the .DefaultIfEmpty

takes over, and it gives us back a null sale. So now we get the current product and a null passed into sale, and we then also build our new product order. And that's why we need to use the null conditional operator down on the sale items to make sure that we're getting back a null or the actual value. So, method syntax, little bit different than the query syntax, but accomplishes the exact same thing. In this module, we saw that the join clause helps us combine two or more collections. Now be careful because the syntax can be a little different from the query to the method syntax. GroupJoin, that method helps us create like a one-to-many object. And we can use SelectMany and DefaultIfEmpty to simulate an outer join. Up next, use the group clause to produce grouped collections. I hope you'll join me for the next module.

Use the Group Clause to Produce Grouped Collections

Grouping Products by Size

Hello. Paul Sheriff with Pluralsight. This module is Use the Group Clause to Produce Grouped Collections. The goals for this module are to learn how to group data with the two different syntaxes, of course, with the query and the method syntax, but there's also a couple of different syntax is even just within the query syntax. We're going to show you how to use order by two different ways, we're going to simulate a SQL HAVING clause, and we're going to create some grouped subqueries. When we talk about grouping, I'm talking about grouping products by size, for instance. So in our ProductRepository, we might have a bunch of products. Some of them are size 58; some are size medium. What I want to do is I want to have the size of 58. And then underneath that, I want to have a collection of those products that fall within that size. Then I should have another grouping of another record that would be the size of M and, of course, the M products collection under that that match size M. The way we do this then is using the group and the by keywords, and we group by a specific property in the object, so in this case size. And we can also order by the same property as well. And this creates what's called an IGrouping collection. So let's take a look at our first example of doing a group by. Open the start folder for this module, and then open the SamplesViewModel class and locate GroupByQuery method. What you'll see is we're now returning a list of an IGrouping interface, and we pass this generic interface two parameters. The first one is the data type of the key that we're grouping on, and the second one is the collection type. So in this case, it's going to be a collection of products that fall within that key. We load all the product data. And then, on line 16, we start writing list = from prod in products. I'm going to order by the product.Size. I'm then going to group prod by prod.Size. So here's where we're doing the grouping by size. Now, it'll

make more sense if we set a breakpoint here after writing our query and we run this. So go to Program, do the GroupByQuery method, and let's run. And what we'll see here in the list is our IGrouping interface where the key is null. There are three products that fit within that collection. Where the key is a blank, there's only one product. Where the key is 42, so these are all the sizes. We have a null. We have empty sizes. We have a 42 size. We have a 44 size. And then you can see each of the collection of products under each one of these group items. And I ordered them by size before I did the grouping. So we can go ahead and just let this run, and we will see the same thing appear here in our console window. If we come back into the SamplesViewModel and we locate the GroupBy method, we're doing the same thing just using the method syntax. So in this case, it's list = products.OrderBy the size .GroupBy the size. We get the exact same results.

Ordering by the Key Property

In the last demo, I showed you ordering the data prior to grouping. There's another way we can do it where we flip it around and we do the ordering after the grouping. But to do that, we're going to order by a key property. Now for the query syntax, we have to use into and select to make this happen. Let's take a look. Locate the GroupByUsingKeyQuery method. Inside of here, if you look at the query syntax, I've added a couple of new things on. So now list = from prod in products group prod by prod.Size into sizes. This creates a new variable that is now organized by size. So we can order by sizes.Key. Remember what I said on that IGrouping interface. The first parameter that we pass into that generic is a string, and that's the data type of the key. Well, there's a key property. So it takes all the size values, puts them into the key property. So we're ordering by that. And then we just have to simply select sizes to return that IGrouping list. Let's go ahead and try this one out. And we see the results are exactly the same. So no change, just an alternate way to accomplish the same thing. If we come back over here and we take a look at how to do this using the method syntax, well, it's actually really easy. As long as on line 72 you do list = products.GroupBy, you do that method first, then you pass to the order by that sizes, and we have now the sizes.Key that we can sort on and then select the sizes. And that gives us again our list of the IGrouping interface.

Filtering the Grouped Data

In SQL, we can apply a having clause after the grouping, and that allows us to filter the grouped data. For instance, we could say I only want to see those sizes that have a count of products greater than 2. So for example, if I have a size of large and it only contains one product, then I don't want to see that size in the final result. But any other sizes where it has greater than 2, so three or more products underneath that size, let's include those in the final result set. So, let's

take a look at a demo where we apply the where expression after the grouping, which simulates a having in SQL. Locate the GroupByWhereQuery method, and let's write the syntax that you see here, list = from prod in products. I'm going to order by prod.Size and then group prod by prod.Size into sizes. We have to use the into because now in the where clause, we want to say where sizes.Count is greater than 2. So it's taking a look at that whole IGrouping collection, and then it's looking at each product collection underneath the size and saying is the count of those greater than 2? If it is, then include that in the final result set. Let's copy this, and let's run this method to see the final result. Now what we end up with, as you can see here, is just a shorter list of that collection. So the size of empty is the count of 43, size of 44 is a count of 6, size of 48 has a count of 5, etc. So we got only those groupings of size where they have more than 2 products. Now let's take a look at doing the same thing using the method syntax. So locate the GroupByWhere method, grab your products, and let's write the syntax, list = products.OrderBy p.Size GroupBy the size Where sizes sizes.Count is greater than 2, select those sizes. So that gives us back our final grouping.

Creating a One-to-many Using a Subquery

Let's now look at creating a subquery. What we're going to do now is we're going to create a collection of products for a particular sales order. Remember before, I had the product and then I had a collection of sales underneath it. We're kind of going to flip that around now so it's very similar to the group join. But what I'm going to do is add another LINQ query within the select. So let's take a look at creating a grouped subquery. Locate the GroupBySubQueryQuery method. What we're going to do this time is we're going to return a SaleProducts list. So if we go down here under the DataLayer and we look under CompositeClasses, you'll find the SaleProducts class. So this time it's a SalesOrderID and under that is a list of product objects. So I have a Products property, and I have a SalesOrderID property. So we grab all of our products. We grab all of our sales order data. And now let's write the query syntax, list = from sale in sales orderby the SalesOrderID. Then let's group the sale by the SalesOrderID into newSales. We're going to select a new SaleProducts. So we're going to create a new SaleProduct each time through as it moves through the list of sales. The SalesOrderID is going to be the newSales.Key because, remember, that's a grouping that we've got. And then, look at products. The products property is going to be assign equal to a subquery from prod in products orderby the ProductID. Join the sale in sales on the ProductID where it equals the sale.ProductID. So now we're only getting those products that match to that sale where the SalesOrderID is equal to the newSales key because this newSales key is the SalesOrderID, and then we select the product. So that builds our collection of products based on the key that we found in our grouping. Let's go ahead and try this out. I think what I'm going to do is set a breakpoint here again because it's I think very effective to show it

sometimes here. Let's copy the method over into Program, and let's go ahead and run. And if we hover over list, we now see that we've got our products, our SaleProducts. So it has the SalesOrderID and then how many products are attached to that SalesOrderID? We'll just let it run, and then we'll be able to see all of the data spit out here into the console window. Let's go ahead and go back over to SalesViewModel and look at how we would do this using the method syntax. So locate the GroupBySubQuery method, grab your products, grab your sales. On line 166, let's start writing list = sales.OrderBy. We're ordering by the SalesOrderID. We're grouping by the SalesOrderID. Then we're selecting newSales pushed into that's the collection, the grouping, new SaleProducts. And we're grabbing the key from newSales, that grouping, and assigning that to the SalesOrderID. The products is going to be equal to products.OrderBy, ordering by the ProductID, joining on the newSales. Our key elements here are the ProductID in the product and the ProductID in the sale. We then pass that to that third parameter, which is an anonymous function. And in this case, all we have to do is return that product object, and that creates our new collection of products.

Simulate Distinct() Using Group By

I wanted to show you one more little demo here that is simulating the Distinct method. So let's say we wanted to get back distinct colors like we showed you before. We could use the group by combined with FirstOrDefault. And in this way, we can actually show you how to simulate Distinct. Locate the GroupByDistinctQuery. Grab a list of products. And now let's write our query syntax, list = from prod in products orderby prod.Color. So we're ordering by the color. Then we're going to group prod by prod.Color into groupedColors. So we now have a collection of grouped colors. And now we can select groupedColors.FirstOrDefault .Color. And that gives us a string, and that's what we've said we want to return from this as a list of strings. What is happening here is you get a grouped set of products by color. Applying FirstOrDefault gives you the first color in each grouping. Let's run this, and we should see exactly like what we saw when we did the Distinct query. We get just our six colors. And we can do, of course, the same thing using the method syntax. So locate the GroupByDistinct method and write your method syntax list = products.GroupBy color select groupedColors.FirstOrDefault .Color and an order by the color. In this module, we talked about grouping and how it can create a one-to-many structure. Now the order by is optional, and we can order before or after the grouping. We can filter the group data using the where expression, so very similar to a having in SQL. We also saw how you can build your own grouped subquery, and we even saw how you could simulate distinct. Up next, aggregate data in collections. I hope you'll join me for the next module.

Aggregate Data in Collections

Using Count() and Filtering the Count

Hello. Paul Sheriff with Pluralsight. This module is Aggregate Data in Collections. The goals for this module are to use the Aggregate functions such as Count, Min, MinBy, Max, MaxBy, Average, and Sum. We're also going to use the Aggregate method, and we're going to aggregate by a group of data. And I'm going to show you how to make that aggregating by a group of data a little bit more efficient. All of the Aggregate functions are going to calculate a single value from a property in a collection of objects. Now the Aggregate method is always applied after the query executes, which means it's already gotten the whole collection by applying all the where clauses and everything. It retrieves the list, and then the method is applied. Let's take a look at a couple of demos using Count and counting using a filter. Locate the start folder for this module and load the LINQSamples project. And then, open SamplesViewModel.cs and locate CountQuery method. We're going to get the list of products on line 13, and then we're going to write the query syntax, value = from prod in product select prod. So that gives us a list of products to which we then apply the Count method. And that counts all of the objects in that particular collection and returns that value. If we make sure this is the one we're running, we can see there are 40 products in that collection. If we come over here and we scroll down to the Count method, we can see the method syntax is really simple. It's basically value = products.Count, and that's a property now instead of the method. Let's also take a look at how we can filter this. If you scroll down to CountFilteredQuery, look at what we're doing. We've got two different syntaxes that we can apply here. We can say value = from prod in products select prod and then .Count, passing in the prod and then prod.Color is to red. So we're putting the lambda expression inside of the Count method. Or we can actually use the where clause on the from and then just simply apply the count. So either one of these works with the Count if you want to filter the records. And if we go down to the CountFiltered method, we can see the exact same thing here on line 75 where we do products.Count and then pass in our lambda expression. Or we can do value = products.Where, passing in the lambda expression to search for color red and then apply the Count method. You can choose which one you want to use.

Using Min() and Max() Methods

The next aggregate functions we're going to look at will be Min and Max. Locate the MinQuery method. Notice on line 90 we're now going to be returning a decimal value because what we're going to do on line 95, we're going to say value = from prod in product select prod.ListPrice and ListPrice is a decimal. And then we apply the Min method to that to give us the minimum list price in the collection. Now the alternate query syntax you can use is the normal from prod in product select prod. So you get the list of all of the products, then pass your

lambda expression to the Min method, prod and an arrow syntax prod.List Price. So you can do either one that you want here. The Min method looks exactly the same, line 115, value = products.Select p arrow syntax p.ListPrice. So get the list price as a collection and then apply the Min method to that. Or you can simply use products.Min and pass in that expression, which is saying the list price is what we want to get the minimum of. If we go ahead and we come over here and we do MinQuery and we run this, you can see that the minimum list price in our collection of products is $8.99. Back in SamplesViewModel, locate MaxQuery. Now this is exactly the same. We have the same query syntaxes here as we did with the Min function. You're simply applying the Max method to that list of products and, again, the list price on those products. You can either choose query syntax number 1 shown on line 135 or query syntax number 2 shown on line 138. If we try this out, we now see the maximum list price that's available in our products collection. And go back to the SampleViewModels and take a look at the method syntax on line 155 where you use the Select, the ListPrice, and the Max or simply do value = products.Max, passing in the p.ListPrice.

Using MinBy() and MaxBy() Methods

Min and Max return a decimal. But what if you wanted to return the actual product object that has that minimum list price or that maximum list price? We can do that using the MinBy and the MaxBy LINQ methods. Let's take a look. Locate the MinByQuery method. And now on line 175, we can say product = from prod in product select prod.MinBy and pass in your expression of what you're looking for, so in this case the ListPrice. Notice that this is returning a product object. So now when we run this one, instead of giving us just that decimal value, it's now going to give us the complete product object that says which one has the lowest list price. Now, if there are two or more that have the same list price, it's going to do like a FirstOrDefault. So it will just return one to you. And if you scroll down to the MinBy method, you can see that we have the method syntax right here, product = products.MinBy and then pass in your expression to get the ListPrice. Next, scroll down and find MaxByQuery. And on line 209, exactly the same as the MinBy, except now we're doing MaxBy. And once again, it will return the product that has the maximum list price. And again, if there's more than one that has the same list price that's maximum, it does a FirstOrDefault and returns that to you. We can also do the same thing using the method syntax product = products.MaxBy, passing in your property name, in this case ListPrice, that you want to get the maximum by.

Using Average() and Sum() Methods

So I think you're starting to see the pattern for these aggregate functions. They're all very similar. So let's take a look at Average and Sum now. Locate the

AverageQuery method, and we again have two different syntaxes that we can use. So the first one value = from prod in products select prod.ListPrice that gives you a collection of decimal values, and then it finds the average of those. Or we can actually do value = from prod in products select prod, get the list of products, and then apply the average and tell it which property you wish to use to get the average of, in this case the ListPrice. Let's copy this over to Program, and let's run this. And we now see the average of all the list prices in our products collection. And of course, we can do the same thing, two different syntaxes using the method syntax, value = products.Select p arrow syntax p.ListPrice and then apply the average or pass in the key expression, here p.ListPrice to the Average method. Next, locate the SumQuery method. This one we're going to calculate the sum of all the list prices in the products collection. We have two different syntaxes, value = from prod in products. Select prod.ListPrice gives you the collection of list prices and then sum those. Or value = from prod in products select prod, get the list of products, and then apply the sum method, passing in our expression that says what property to use, in this case ListPrice. Let's copy this one in, run this, and now we see the total sum of all of the products in our products collection. We also have the corresponding method syntax for this as well, so line 303. We do the select on the ListPrice and then sum or, on line 306, products.Sum, passing in the property ListPrice to the Sum method.

Simulate Sum() Using Aggregate()

Now obviously, each of the aggregate methods that we just looked at are iterating over the collection and then doing some operation. Well, what if you wanted to do some sort of operation that is custom, not a sum, not a min, not a max, but something that you need to do? Well for that, Microsoft has given us the Aggregate method, which iterates over the collection. You then supply a custom method for doing whatever calculation you want, and then it returns a single value. So as an example, let's start out and just take a look at we can simulate the Sum method using the Aggregate method. Locate the AggregateQuery method. We're still going to be working with our products collection. We're going to be returning a decimal value. So we're going to do value = from prod in products select prods. That gives us our list of products. And then we apply the Aggregate method. Now the Aggregate method here, we pass in the seed value. Now the value is going to be a decimal, so I use 0M. Then the second parameter to aggregate is our function that we're going to use to do whatever accumulation we're going to need to do. So in this case, I'm going to pass in sum comma prod. So it's passing in sum, which is in this case the seed 0M for the first time through, and it passes the first product. Then we use arrow syntax to say sum plus equals prod.ListPrice. So the first time it's 0 plus the first products ListPrice. Then the second time through it takes that sum, passes it back to that function, passes the next product, and then does the same thing again. So by the

time we're all done, we end up then with the sum just like we saw before using the Sum method. Let's go ahead and take a run here and just validate that. And we see it does come up to 26,274.41. That's exactly what we saw when we did the Sum method. All right, so there's the query syntax for that. Let's take a look at the method syntax. Well, as you'd expect, pretty much the same thing, value = products.Aggregate 0M comma and then our function that we use to accumulate, and that returns that same value.

Using Aggregate() Method with a Custom Expression

Now obviously, you're not going to duplicate the functionality that's already built to LINQ. So let's take a look at using this Aggregate method but using our own custom expression. Let's take a look at the AggregateCustomQuery method here. First off, we're going to load up the sales order data now. So we'll have a collection of sales order data. Again, we're going to return a decimal value. Down on line 362, you're going to write value = from sale in sales select sale. So we have our collection of sales. Now I'm going to apply the Aggregate method. I'm going to seed it with 0. So it's the same thing. We're going to pass in the sum and the sale object. And then look what I'm doing. I'm doing sum plus equals, and then my expression now is sale.OrderQuantity times the sale.UnitPrice. So really what I'm looking for here is a total of all the sales in my sales object. So let's take a look and see what this comes up to be. So it iterates over everything, applies that expression, and then it calculates for all the sales what is the quantity times the unit price. Let's go back down here to the AggregateCustom method, and this one shows you then the same thing just using the method syntax, which really just eliminates that from in the select. We're just using the sales collection.

Using Grouping with Aggregation

Another thing we can do with aggregation is we can group things and aggregate at the same time. So what we want to do now is we're going to take and group products by size. And then for each size, we're going to calculate the count, the min, the max, and the average. So we'll end up with a collection of objects that look kind of like this where we have size L, the count is 3, the min is 8.99, the max is 157.55, the average is 65.68. These are all just examples. And then maybe for the size medium, we have a count of six and a min and a max and an average. So we keep going and we aggregate while we're doing the grouping. So let's take a look at aggregating over a group of data. Locate the AggregateUsingGroupByQuery method. We get all of our products, and look what we're getting back. We're getting back a list of ProductStats objects. So if we were to go take a look at this, you'll find it down in the LINQSamples.DataLayer underneath CompositeClasses, and this is called ProductStats. Now this class ProductStats has a size, has a total products, total list price, minimum list price,

max list price, and average list price. So these are the fields that we're going to be filling in. If we come over here and we take a look at our query, list = from prod in products groupby prod by prod.Size into our size group. So there we go. We've got our size group, and we only want to get those where the sizeGroup.Count is greater than 0. So we've seen all of that grouping syntax before. But now what I'm doing is I'm selecting new ProductStats. So I'm creating a new one, filling in the size property with sizeGroup.Key, so that gives me the size. Then I do total products = sizeGroup.Count, MinListPrice = sizeGroup.Min, passing in the ListPrice, MaxListPrice = sizeGroup.Max, passing in the ListPrice, and AverageListPrice = sizeGroup.Average. So what it's doing is for each one of these groups, it's calculating the count, the min, the max, and the average, filling in the appropriate properties. I then use into result because I want to order by result.Size and then select that result back. Let's take a look at what this does. Let's copy this, go over to our Program, and let's run it. So what we see now is we get 15 product sizes back. If we go all the way to the top here, we can see the first size is either a null or an empty. It's hard to tell here. But then we have a total products of 3 where a minimum list price is 3499 maximum list price is 3699, giving us an average of 3599. Let's scroll down and take a look at another one here. Size 44, total product 5, minimum list price 337.22, maximum list price is 1431.50, giving us an average list price of 902.48. So all of this was done using this very simple query syntax that you see right here. And of course, we can write the same thing using the method syntax. So on line 429, list equals products.GroupBy sale sale.Size select the sizeGroup, create new ProductStats, do all of the same property setting that we saw before and then order by the result.Size.

Making Aggregation More Efficient

The code we wrote works. It gives us back the correct data. But there's a problem. It's just not very efficient. And the problem is is that each of the following lines of code that we wrote is looping through the entire group collection each time. So it's looping to get the count, it's then looping to get the minimum, it's then looping to get the maximum, and it's looping to get the average. Obviously, this is very inefficient. So, what we want to do is learn how to do a little bit more efficient group aggregation. And the way we're going to do that is we're going to create a class to hold accumulators. So it'll be one property for the count, the min, the max, etc. We're still going to use the Aggregate method, but we're only going to let it loop over the data one time to gather the statistics. And it's because we're able to use this accumulator class that it can do that. Let's take a look at a more efficient aggregation by using extension methods. Before I show you the query, let's take a closer look at the ProductStats class. This is also going to be my Accumulator class. Notice the constructor. It initializes all of the variables to a good default start value, so each of the properties that we saw before. If we take a look at the Accumulate method, this is what's going to be called every time through the aggregation of going over the

products for each size group. So it'll pass us a product. We then increment TotalProducts by one. We increment the ListPrice by the ListPrice in the product. We then calculate the MinListPrice by doing a Math.Min. So whichever one's lower, the MinListPrice or the current product's ListPrice, it'll put that into the MinListPrice property and same things for the maximum. And then it returns the ProductStats object itself. We also have one more method called ComputeAverage. This is called at the very end. After all of the grouping has been done and the collection has been iterated over, we now can compute the final average for that group. So average ListPrice equals Total ListPrice divided by TotalProducts. So how is this called? If you go back to the SamplesViewModel and you locate the AggregateMoreEfficient method, on line 430, we can start writing our method syntax, and it only works with method syntax. You cannot use the query syntax for particular scenario. List = products.GroupBy sale sale.Size Where the sizeGroup.Count is greater than 0, so very typical grouping scenario. What we do now is we do the select on that size group, and we create an anonymous function in here. This is something that you can't do in the query syntax. So you see the open curly brace. And then within here, I'm going to create a new accumulator object. So remember this happens for each size group. So I've got the size, and I have a collection of what, a couple of products or two or three or six products, however many. I create one ProductStats object for that group key, and I put that group key into the size. Then, on the sizeGroup, I call .Aggregate. So it's going to iterate over though that collection of products. We pass in our accumulator. The accumulator is then in the second parameter passed in along with the current product as it's iterating over the collection of products. And we then call the accumulator's .Accumulate method that we just looked at, passing in that product. So, it increments the TotalProducts, it increments the ListPrice, calculates the Min and the Max in that Accumulate method. Then, it goes and reiterates over the next product and the next product until we're all done for this size group. When it's all done, that third parameter to the Aggregate function is now called, and that's that ComputeAverage. So what it's done is it now has this accumulator object that now has all those values filled in. And it did it doing just one iteration one time through as opposed to having to call it each time like we did before. We then return that accumulated results, and that's part of then what comes back. So what comes back if we run this is we should get the exact same values we did when we did it before, but it's just more efficient in its operation because it only looped through each grouping one time. In this module, we learned to use the aggregate functions Count, Min, MinBy, Max, MaxBy, Sum, and Average. We also learned to use that Aggregate method. We also created a custom class with accumulators because that allows us to build a more efficient method of aggregating groups of data. Up next, use LINQ to iterate over collections.

Use LINQ to Iterate over Collections

Using ForEach() to Calculate a Line Total

Hello. Paul Sheriff with Pluralsight. This module is Use LINQ to Iterate over Collections. The goal for this module is to show you how to perform set operations upon a collection. We're going to do that by showing you how to assign values within a collection using nothing but LINQ. Obviously, LINQ is a set operation, which means it iterates over the entire collection, and it allows us to set a property value in a collection. It's kind of like a SQL update if you will. And some examples is maybe you might want to calculate a LineTotal or set the total sales for a product. Or maybe you want to set the length of a name property. So for example, with the query syntax, you would do from prod in products let tmp, t-m-p, = prod.NameLength = prod.Name.Length. So if you have a property called NameLength in your product object, you could actually set it using this little temporary variable trick. And you only have to do that in the query syntax. Using the method syntax, you use the ForEach. And for this one, you just simply do the assignment. So p.NameLength, so you're setting that property, equal to prod.Name.Length. So assuming that the product name is a string, you're getting the length of it and just putting it into another property within the product object. So this is just one idea, but let's take a look at a more real-world example where you have sales and you want to calculate the line total by getting the order quantity times the unit price. Open the start folder for this module and open up SamplesViewModel and locate the ForEachQuery method. We're going to use sales data, so we'll get all the sales data, and then we'll write our query syntax. Now notice we're not assigning the results to anything because we're just doing an in-place update. So we do from sale in sales. And when using the query syntax, you must use a let tmp, you have to assign to some throwaway variable, equals and then here's where you do your actual assignment, sale.LineTotal equals sale.OrderQty times sale.UnitPrice. So it calculates OrderQty times UnitPrice and assigns it to the LineTotal property in the sales collection. We select sale, and then we return that. So make sure your ForEach query is the one that we're doing here in Program, and let's go ahead and run this. And when it comes back, we should see all the totals are there. They're all calculated correctly. So if we look at the quantity of 1 and a unit price of 1431.50, you see the line total is 1431.50. If we go down to the next one, we see quantity of 10 and a unit price of 8.99. The LineTotal now comes out to be 89.90, just as we'd expect. If we go back and we find the ForEach method, in this one, the method syntax, we actually use sales.ForEach. And then inside of here, we don't have to use the let. You simply pass in the sale for each time through the loop and you perform the calculation of sale.LineTotal equals sale.OrderQty times sale.UnitPrice.

Using ForEach() and a Sub-query to Calculate Total Sales

Let's take a look at another example, and this one is we're in the product object, and we have a TotalSales property, and we're going to use a subquery into the sales collection to calculate the total sales into our new property in our Product class. Let's take a look. Our Product class has all the typical properties. But then if you look down, I have a little section called Calculated Properties. I have a NameLength and I have a TotalSales. Now that NameLength I showed you in the slide. We're now going to be using TotalSales, and it is marked as a nullable data type because I want to be able to either have null or the total sales in there. Go back to SamplesViewModel. Locate the ForEachSubQueryQuery method. We're going to get all product data. We're going to get all sales data. And now you write your query syntax, from prod in products let tmp, so that's just any variable name you want to make up, equals prod.TotalSales. And that's going to be equal to sales.Where sale sale.ProductID = prod.Product ID. So it finds those sales that match on the current product. We then apply the Sum method using the LineTotal. We select the prod, and we get back our list. So if we copy this now into here and we run and we start the debugging process, we should now see, if we go all the way to the top, we now see a Total Sales there. And we can see it's on some and not others, and that's because some products have sales and some do not. If we go back to SamplesViewModel and we look for the ForEachSubQuery method, we again gather all the product and sales data and then we do a products.ForEach, passing in our product each time, p.TotalSales = sales.Where on the ProductID, where they match up and then sum the LineTotal.

Call a Custom Method from ForEach()

Instead of performing that subquery right inline, you could set the total sales using a method as well. So you're allowed to call a method inside of the query. Let's take a look at that. And then we're going to show you how you can filter the results on total sales to make sure you only get those products that have a total sales greater than 0. Locate the ForEachQueryCallingMethod query. We're going to get our products; we're going to get our sales. We're now going to write a query, var list = from prod in products let tmp = prod.TotalSales = CalculateTotalSalesForProduct, and we're going to pass in the current product and the sales collection. So the CalculateTotalSalesForProduct method is this one right here. You can see it's going to return a decimal, CalculateTotalSalesForProduct. It accepts a product object and the list of sales orders. And then we run the same thing that we had before, sales.Where where the ProductID and the sales is equal to the ProductID and the product and then sum the LineTotal. So that hasn't changed. The reason I broke this out is to show you that you can call a method from within your query, and there may be some more complicated logic that you need to do inside of here. So, you call that method. It sets the total sales. Now what we get back from this is an IEnumerable of Products. It's not a list of products. It's an IEnumerable because I didn't apply

the .ToList. It really has run this, but it's still just an IEnumerable. Since it's an IEnumerable, I can do a list = list.Where, and I can now filter that ending list here, that the product TotalSales greater than 0. And then I returned the list.ToList. So let's make sure this is the one we're calling over here. We'll start the debugging. And we now have gotten the total sales by calculating those from a method, and we've also gotten only those with a total sales greater than 0. If we come back and we take a look then at the same thing using the method syntax, ForEachQueryCalling method, we get our products. We get our sales, Products.ForEach. Calculate the total sales by calling that same method. And then, products = products.Where, applying TotalSales greater than .ToList, we can now return those products. In this module, we use the ForEach method to iterate over the collection. When you're using the query syntax, make sure you use the let keyword to assign values. Remember, it's just a throwaway value. We then can use subqueries to calculate values, and we can even call methods from within our ForEach. Up next, understanding deferred execution, streaming, and non-streaming operations. I hope you'll join me for the next module.

Understanding Deferred Execution, Streaming, and Non-streaming Operations

Classification of LINQ Queries

Hello. Paul Sheriff with Pluralsight. This module is Understanding Deferred Execution, Streaming, and Non-streaming Operations. The goals for this module are to learn the different LINQ execution methods and there's four of them, deferred, immediate, non-streaming, and streaming. We'll take a look under the hood of these streaming methods to see how they work. I'm also going to show you creating a custom filter extension method that shows you non-streaming. Then we'll create a custom filter extension method using the yield keyword, which is an example of streaming. As I mentioned, there's these different types of LINQ execution. When we talk about deferred execution, which is the focus of this particular module, we're going to see two things. We're going to see streaming operations and non-streaming operations. Then as I mentioned, there's one more and that's called immediate. And that means the query is just going to be executed and you're going to get the data back right away. Now there's a great Microsoft site that shows you these different LINQ operator classifications. Let's go ahead and take a look at this. Microsoft has this great web page that talks about the classification of standard query operators. And if we take a look, it shows you the same ones I talked about, immediate, deferred, streaming, and non-streaming. I would highly recommend that you read this page. Then it's got a nice little table here that shows you each of the different LINQ operators, and many of these we have gone through in this class. Tells you

what the return type is. But more important, it tells you whether it does immediate execution, deferred streaming execution, or deferred non-streaming execution. And it has a list of all the various LINQ methods here and their classification type.

Deferred Execution, Streaming, and Non-streaming

Let's talk about deferred execution. When we create this LINQ query, it's actually just a data structure that is ready to execute. The query is not really executed until a value is needed. So the execution would happen if you do the ForEach, if you do a Count, if you do a ToList, which, as you've seen mostly throughout this course, that's what you have been doing is a ToList. So when you apply the ToList, that query is executed and the results are returned to you. Also an OrderBy, and some of the other ones that you saw there on that web page would force the execution to happen. But if we take a look and we create a query just like this where I've got a data structure now, IEnumerable Product query = from prod in products where prod.Color is equal to Red select prod. That is not executed yet. It is simply a query ready to be executed. So the query will actually be executed once a value is requested. So for example, if we now say foreach var item in query, it now needs to return an item in order to give it to us to put into this item variable. So that's the time that this query is finally executed. So obviously, immediate execution is the opposite of deferred where the query is executed immediately, and an operator or method that requires all items to be processed is when immediate execution happens, like a ToList or an OrderBy. So if we take a look again at IEnumerable Product query = from prod in Products select prod, it's creating that data structure. But because we're applying the .ToList operator, that query is now executed immediately, and all the data is now filled in to the query variable. Now I mentioned streaming. What are streaming operators? Well, I want you to think about Netflix. Do you have to download the entire movie prior to start watching the movie? No. It simply gives you the movie in chunks. And that's the same thing that we're talking about here. The results can be returned prior to the entire collection being read. And we're going to see a really good example of this in this module, but examples of this are Distinct, DistinctBy, GroupBy, Join, Select, Skip, Take, Union, and the Where because what's going to happen now is both the Select and the Where in this particular example are deferred in streaming. If they weren't, then the products collection would have to be looped through two times, once for the Select method and once for the Where. But we're going to see how this is not the case. It can start returning values because it looks at the Where and says, well, really, I just need to return those that are red, and then we'll give it to the Select. So that's an example of a streaming operator. And of course, the opposite of that is non-streaming. This is where all data in a collection must be read before a result can be returned, and examples of that would be the Except, the ExceptBy, the GroupBy, the GroupJoin, the Intersect, the IntersectBy, the

Join, OrderBy, ThenBy. Each of these must read all of the data in order to perform its operation.

The Classes for Illustrating Deferred Execution

Let's dive into our first demo and look at deferred execution using a foreach. For this module, I'm going to use Visual Studio instead of Visual Studio Code, and I'm using Visual Studio 2022. And the reason why is just because I can show you some things better with the debugger in Visual Studio. But feel free to follow along with Visual Studio Code, or you could download Visual Studio 2022 Community Edition, which is free. Go to the LINQSamples.DataLayer, expand the EntityClasses folder, and bring up ProductSimple. This class simply has two properties, Name and Color, and they're full property gets and sets. The reason I'm doing that is because I wanted to be able to put into the getter the Console.WriteLine Product Name. And in the color getter, I wanted to do color .WriteLine Color is Color for that particular product name. And this is going to help us illustrate the deferred execution. I'm also overriding the ToString that you see here, and I'm returning From Query _Name, and this will tell us when it's actually being got from the query as opposed from when it's actually getting it from each individual time through the collection. Also, I've created under the RepositoryClasses folder a ProductSimpleRepository. It has a GetAll just like we've seen before, and this one simply returns a new list of ProductSimple objects. And I've just done four of them. That will be enough to illustrate deferred execution.

Illustrating Deferred Execution Using ForEach()

Now go up and locate the SamplesViewModel. And in the SamplesViewModel, I have a deferred execution method. Inside of this method, I'm going to create an IEnumerable of ProductSimple classes called query. I'm going to load the product data by calling the ProductSimpleRepository.GetAll, and then I've got a Debugger.Break so we can stop the program and see things happening underneath the hood. So, let's make sure that this is the one that we're calling here in Program and let's run. And here we are. We now have a list of products. So there's four of them like we saw. I stopped the program and hit F10 here. Now, look down. I have the Autos window open. Under Debug, Window, just choose Autos. Once you have this open, look down there and you can see that the products has a count of four. The query so far is null. I've now hit F10 and query = products.Where, p.Color is equal to Red. But if you look, the query doesn't show a count. It's simply an IEnumerable where list iterator at this point. And you can even see that the results view says expanding the results view will enumerate that IEnumerable, which means it would actually go through it. There's also a property here called current, and it's currently null. So let's watch. So at this

point, the query has not been executed. I'm going to hit F10. Things still haven't happened. But now when I hit F10, so now what it's doing is saying, okay, I'm going to that query, and I'm saying I need to retrieve one into that product variable. That's a ProductSimple type. And now, look at current. Current now does have that current product. Things are starting to happen. So we then execute the Console.WriteLine on that product. It executes the ToString. We then come back up. We go to one more here. The query now has changed to current. See now it's the next one in the list. And we can then keep going through. We went through only twice, and the reason why is because there were two items in that products collection that had a color of red. So it spit out just those two items. Now I'm going to go ahead and press F5 here and let this run, and we will now see our console window. Let's talk about what's going on in here. Now you see why I put in all those Console.WriteLines because now look at this. Let's take a look here at what happened. So now we get the first line, Color is Red for Sport-100 Helmet. That came from the property get, which happens when the first object in the collection is checked to see if it is red. Then, if that color is red, so it matches the criteria, that one product is now returned and we go into line 27, which does a Console.WriteLine on the product variable, which executes the ToString method, and that's where you see the from query Sport-100 Helmet. It then goes back up to the top of the loop, looks at what's next in the query, and that was that current variable that we kept seeing. It then says, okay, the current one is this. Check it against the predicate, the p.Color is equal to Red. It says color is black. Doesn't match so it goes back up to the top of the loop and gets the next item in the list. This one, it accesses the color property. We get the Console.WriteLine, Color is Red for Long Sleeve Logo Jersey. It matches the criteria. It goes into the Console.WriteLine on product, calls the ToString. We now see the From Query: Long Sleeve Logo Jersey. And then it goes to the top of the list one more time, checks the next item in the list, says color equal red. No. But I still get the get. It still had to access that. So the Color is Silver is printed out, but we get no from query and then we are done. Pretty neat to see kind of what's happening under the hood here with this where method.

Step-through Demo of Deferred Execution

Now to really kind of bring this home, let's actually step through that exact same code and we'll see kind of how each thing happens as we move through this code. Go ahead and run this again. And now, what I'm going to do is I'm going to use my F11 to step through the code. So I hit F11. See, nothing happened, right? Nothing was executed there. Now let's hit F11 here and look. As soon as it said, hey, I'm taking something out of the query and I'm going to put it in ProductSimple. The question becomes, am I putting anything in? Well, in order to know that, it has to execute that predicate, that lambda expression up there that says p.Color is equal to Red. So it looks at the color red, I'm going to hit F11 here, and it says, hey, we got one. So it now says I've got a product. Hit F11. We

now jump into the ToString because that's what it's going to call, and that's how the from query appears in the console. Now I could set a breakpoint in the getter, and we could actually see it jump into the color comparison, but I think you get that. Now it's going to the in again. It says okay, what's the current one in the color right now? Oh, it's black. Does that equal red? No, it does not. So I hit F11. It went to the next one. Didn't even go in to the foreach portion. It didn't go in to do the Console.WriteLine. It says nope. I need to skip over that one. See, so it's really just kind of executing this query behind the scenes a little bit. Now we're on the next one that is a red. So it takes it, puts it into the product, spits that out with the ToString there, and Console.WriteLine has been executed. It now comes back up. Says what's the next one? Well, this final one here should be silver. Doesn't match. It executes out of the foreach. Hit F5. We are done, and we see how everything happened. Now, you get a little bit different results when you're stepping through. Don't worry about that. You get the idea. But I wanted to show you that it's really neat. You can kind of understand a little bit about what's going on underneath the hood simply by using the debugging tools here to step through the code.

Using IEnumerator and GetEnumerator()

I've been using this word enumerate. We're enumerating over this list. What does that mean? Well, the foreach is nothing but an enumerator so I want to show you what that looks like in .NET code. Scroll down to the DeferredExecutionEnumerator method. The first few lines of this are exactly the same as what we have, but look down on lines 52 through 56. This is where it's just a little bit different because what we're doing now, this is the same equivalent code as the foreach. I'm doing an IEnumerator ProductSimple called enumerator, and it's going to be equal to query.GetEnumerator. So think of the GetEnumerator is like a little pointer that points to each item in the list. At the very beginning, it's before the first item in the list. So you do a while enumerator.MoveNext. So that moves it to the zero position in the list. So then what does it do? We can actually do a Console.WriteLine enumerator.Current. Remember the Current property that we saw earlier? Now we're going to take advantage of it and use it. So we're just going to spit that out. So this works exactly the same as the foreach. Let's go ahead and copy this into Program.cs and run this. And let's go ahead and step through here. So we get the enumerator, and we can look at this in the Autos. So you see the current is null. But when I do a MoveNext, it moves to that first item. So it then can do an enumerator.Current and spit out that current item, which basically does a ToString on the product object. We can then MoveNext, and we're done. We got two just like we did before because each time through it's evaluating that predicate to find out whether or not it matches the criteria and should be spit out into the final results.

Show Streaming Nature of Where() and Take() Methods

So we've seen how the Where clause still iterates over everything, but it only returns those back into the foreach that match the criteria. But let's show you where the real power of this comes in with the streaming nature of Where combined with the Take method. Locate the method UsingWhereAndTake. We've got the first three lines are the same, but look at the LINQ query now. It's query equals products.Where, prod.Color is equal to Red. Now we're applying the Take and using 1. So I only want the first one that matches the color red. What I want to see now is what happens when I actually try to run this and how it differs from just the simple Where. So take this, paste this into the Program.cs, and run. Now I'm going to go ahead and just let it run. Hit F5 here. Much less data is accessed because it hits the first one, Color is Red. Then it spits that out. And then it says now wait a minute, we have a Take applied here as well. I don't need to go on and do anything. That's really powerful. Now let's go change something though. Let's go down to the ProductSimple repository, and let's change the first color for the first product because that's the one that it just spit out. Let's change that color to a white. Now let's run this. And again, I'm just going to let it run. Now you can see what's happening. Now it is still iterating over each item until it matches that criteria. Once it matches it, now it says, wait a minute, is there a Take out there by the way? And it says, yeah, I got a Take and it's Take(1). Excellent. Here's the final result. So it's going to iterate until it matches that condition, and then it'll apply the Take and say, hey, have we satisfied the Take and the Where conditions?

Create Custom Filtering Extension Method

Hopefully at this point, you're getting a feel for this deferred execution. To really drive it home, I'm going to do a couple more examples now. I'm first going to give you a simple filtering extension method so we can start taking a look at how we might build one of these streaming operators ourselves. I'm going to go back to the ProductSimpleRepository and fill that color back in as red. Now I want to show you under LINQSamples.Common, HelperClasses, this LINQ helper class. It's a public static class. It then has a public static IEnumerable of T method called FilterSimple. We're going to pass to it the source, which will be an IEnumerable T, which in our case will be that IEnumerable ProductSimple. And then we also pass in a function that's going to be our predicate. It's a function of T, and it returns a Boolean. Then on line 19, I'm going to create my result list. It'll be an empty list. Then I simply iterate over the source. So for each var item in source, if the predicate is true, then add the item to the result. And then at the end, I finally return the resulting list. Let's now go over to SamplesViewModel, locate UsingSimpleFilter method. We're going to get our product data. And now look at line 106, query = products.FilterSimple, prod arrow syntax prod.Color is equal to

Red. So FilterSimple, it looks just like a Where clause. I mean that's what we're trying to simulate here. Products is going to be passed in as the source to FilterSimple, and the lambda expression there is passed into the function of FilterSimple. Let's go ahead and run this and see how this works. Press F10. Now I'm going to use F11 because we're going to come in here. We're going to see the source is our product list. The function, that predicate, is this Boolean. That's going to return a Boolean, color is equal to red. We'll create our empty list. And now we're simply going to iterate over the source, grabbing each product, and then we apply the predicate. I'm going to press F11 here to show that it does go to that color. Checks it out. It says ah yes, that one matches. Let's add it to the result. Goes and gets the next item. It again checks it. Doesn't match. Goes to the next item. Checks it It matches. We add it. We go through it one more time, and we now have our result of two items that match that predicate. And now we can do our foreach and spit that out on the Console.WriteLine. This does not look like the LINQ Where clause, does it? Now it's gone through all four colors first and hit those and then reports it back as the from query; whereas with the Where clause, it actually did one color and then reports it back to the from query. So this is our first attempt at writing our own little simple filter. But let's take a look at what else we can do with this.

Apply Take() to Custom Filtering Method

One of the things I showed you with the Where was adding the Take method on as well. Well, let's take a look at what happens with our custom extension method using Take. Locate the UsingSimpleFilterAndTake. Everything's the same except on line 133, you see I've added the .Take(1) to the FilterSimple. Well, let's go ahead and run this. Now what I'm going to do is I'm just going to go ahead and let it run. I'm going to press F5 here so we can see the results. And you can see the results. We still go through all of the list first, and then it applies the Take(1). So what we're showing here is the non-streaming nature of what I wrote because we are iterating over the entire collection before any other LINQ methods are applied.

Use Yield Keyword to Create Streaming Extension Method

So how do we create a streaming operation so it is very similar to the LINQ Where method? Well, what we do is we use the yield keyword to create a streaming operation. Let's take a look. Go back to the LINQ helper class and locate the Filter method. The declaration for this method is exactly the same as what we had for FilterSimple, but the body has been greatly simplified. I no longer need a result list. Instead, I'm going to iterate over each item in the source. I'm going to check the predicate of that item. And now when I return the item, I'm going to add the

yield keyword in front of that. Now yield says give control back to whoever called me. So let's take a look at how this works. Go back to SamplesViewModel. Locate the UsingYield method. We're going to do everything still the same. Create our query; create our products. But now, in the LINQ query on line 159, it's query = products.Filter. We've got our predicate. Then down below, I've got our foreach. Well let's walk through this and find out what happens. So let's go ahead and run this. Grab the UsingYield here and paste it into Program. And what I'm going to do first, let's just hit F5 and let it run. Well now that is exactly like what we saw for the Where. So we have actually simulated how the Where clause works in LINQ because now we're getting color and then from, color, color, from, and then color. Let's actually walk through this one now. So I'm going to hit F10. I'm going to hit F11, F11, F11. And remember on the in, what it's going to do is it should come down into here, starts iterating, checks the predicate. There it is. It's checking the predicate. And then it yields return item. Now look where it's going to go. It's not going to continue in here. It's going to come back to the foreach and say, hey, I am returning that item for you to put into your product variable. We can then do a Console.WriteLine on that product. We now go to the next in. It now goes back up and says, okay, I will continue with the next item in the list now. And if we hover over this item, we can see it's the next one, that road frame that is black. So obviously, this predicate is not going to succeed. It skips. It doesn't return execution back to the foreach, does it? It now just continues on. Checks the next one. This one does have the correct color. So it now spits this one out, goes back in here, goes through and says, hey, no more. Meet that criteria. It is now done with that, and so this for each is done as well. And look at that, we have created a streaming operation.

Use the Yield Keyword with Take() Method

Now to really see this in action, what would happen if we applied the Take to our custom extension method? Let's find out. Locate the UsingYieldAndTake method. Everything in here is exactly the same as what I just showed you. On line 188, query = products.Filter, using our predicate, .Take(1). So let's see if this matches what I showed you before when we did the Where with the take. I'm going to go ahead and just let this run, and look at that. We only get one value back. Now it's still iterating over everything because it still has to check all of those values against that Where clause. But it knows when to stop, and that's the important thing. And that's the real big difference between the streaming and the non-streaming operations.

Use the Yield Keyword with OrderBy() Method

Now the question becomes what happens if I have a streaming and a non-streaming operation together in the same query? Well, let's take a look at doing

that using our Yield and the OrderBy. In the SamplesViewModel, locate the UsingYieldAndOrderBy method. And now, on line 217, I've added the OrderBy to our Filter method, which is exactly the same as the Where. So our filter is a streaming operation. But if you remember, the OrderBy is a non-streaming operation. Let's go ahead and try this one out. And again, all I'm going to do is press F5 and let this run so you can see the results. Well, now you see with the OrderBy. What it does first is it does that streaming operation. It goes through them all, figures out what are the ones that I should be ordering by. Then, it applies that Where clause, so it then grabs those products that match the criteria. In this module, we saw deferred execution can be an advantage because we get better performance and hopefully less iterations through our collections. Also, be sure to take advantage of yield when you're writing your own custom LINQ extensions. In this course, we saw that LINQ is a great way to query collections as opposed to doing looping. It's very efficient operations for filtering, sorting, extracting, joining, grouping, and aggregating data. And then, be sure to take advantage of your deferred operations. Well I hope you enjoyed this course. For Pluralsight, my name is Paul Sheriff.